

Open source · Engineering · Problem solving

Problems explored. Changes explained.

A GitHub contribution portfolio

Quaylyn Rimer

github.com/killerdevildog

57

Individual case studies

40

Merged pull requests

17

Completed issue records

What this portfolio documents

One page for each public merged PR and each issue marked completed authored by **killerdevildog**, verified on 2026-10-04. Each page explains the problem, why it mattered, the contribution, the approach, and the documented outcome.

Source discussions, review feedback, code changes, and linked resolutions establish the record. Author-reported tests are identified as such; this research did not rerun the upstream projects' tests. Inferred design reasoning is labeled rather than presented as the author's stated motivation.

A completed issue can represent a maintainer fix, existing-feature guidance, or a concluded proposal. Personal repositories and fork merges are labeled explicitly. These pages do not imply employment, project endorsement, or upstream acceptance where the sources do not establish it.

The contribution record

Ordered by completion date, most recent first. Select any case to jump to its page.

Restoring inherited paint-server transforms in usvg linebender/resvg · PR #1135	05
Tracing lost gradient transforms to a regression linebender/resvg · Issue #1134	06
Restoring the cylinder containment test ThePhysicsGuys/Physics3D · PR #28	07
Reporting inverted cylinder containment ThePhysicsGuys/Physics3D · Issue #29	08
Parsing plus-signed float exponents in Flax FlaxEngine/FlaxEngine · PR #4262	09
Keeping sprite offset setters independent Is-Daouda/is-Engine · PR #10	10
Exposing silent float-parse failures on 1e+10 FlaxEngine/FlaxEngine · Issue #4261	11
Reporting an X setter that moved sprites vertically Is-Daouda/is-Engine · Issue #9	12
Returning full-range angles between 3D vectors x-ege/xege · PR #399	13
Restoring true sprite height in crash recovery LibreSprite/LibreSprite · PR #649	14
Diagnosing supplement angles in a vector helper x-ege/xege · Issue #398	15
Tracing square-canvas crash recovery to its source LibreSprite/LibreSprite · Issue #648	16
Correcting SHA-256 padding for edge-case lengths raysan5/raylib · PR #6144	17
Diagnosing silently wrong SHA-256 digests raysan5/raylib · Issue #6143	18
Tokenizing scientific notation in expressions opentoonz/opentoonz · PR #7113	19
Reporting split exponent literals opentoonz/opentoonz · Issue #7112	20
Correcting a sign in polyhedron raycasts xissburg/edyn · PR #156	21
Diagnosing missed polyhedron ray hits xissburg/edyn · Issue #155	22
Correctly parsing scientific notation in shaders DiligentGraphics/DiligentCore · Issue #804	23
Recovering sphere hits when a ray starts inside turanszkij/WickedEngine · Issue #1695	24

The contribution record

Ordered by completion date, most recent first. Select any case to jump to its page.

Making loaded-object iteration easier	25
OpenLoco/OpenLoco · PR #3614	
Integrating role-specific requirements into a proposal	26
killerdevildog/checkmate · Issue #1	
Selecting the correct SX1280 radio driver	27
tinygs/tinyGS · PR #306	
Preserving const correctness in Linux code	28
libuv/libuv · PR #4843	
Exploring spatial hashing for collision detection	29
killerdevildog/GDevelop · PR #1	
Coalescing free blocks during deallocation	30
bkaradzic/bgfx · PR #3593	
Inspecting wheel archives from the command line	31
pypa/wheel · PR #669	
Batching GPU profiler queue removal	32
nem0/LumixEngine · PR #1538	
Reducing terminal screen serialization overhead	33
ArthurSonzogni/FTXUI · PR #1188	
Compiling HLSL shaders on Linux with Wine	34
bkaradzic/bgfx · PR #3578	
Replacing an obsolete configuration example	35
queer/boxxy · PR #279	
Detecting NaN in ray-plane intersections	36
g3n/engine · PR #317	
Designing a contributor-recognition badge proposal	37
badges/shields · Issue #11227	
Avoiding shared mutable ordering context	38
encode/django-rest-framework · PR #9742	
Correcting generated Scalar return annotations	39
opencv/opencv · PR #27620	
Modernizing the OpenRTS build and navigation	40
methusalah/OpenRTS · PR #117	
Updating a Godot scene to a green sphere	41
Amphuse76/GoGetEm · PR #1	
Making maximum walking distance configurable	42
OpenKore/openkore · PR #4070	
Updating the file-upload demo to native async	43
tornadoweb/tornado · PR #3523	
Improving WebAssembly keyboard character mapping	44
matrixcascade/PainterEngine · PR #144	

The contribution record

Ordered by completion date, most recent first. Select any case to jump to its page.

Removing a broken contributor-agreement step playcanvas/engine · Issue #7877	45
Guarding image resizing against invalid dimensions phase1geo/Minder · PR #659	46
Registering existing test applications with CTest MyGUI/mygui · PR #289	47
Adding native ARM64 Linux release jobs raysan5/raylib · PR #5075	48
Updating the Hello World tutorial APIs alekmaul/pvsneslib · PR #312	49
Correcting repeated spelling errors across VirtualBox VirtualBox/virtualbox · Issue #83	50
Refreshing scrollbar marks from shell integration microsoft/terminal · PR #19185	51
Adapting recipient lookup for newer GPGME profanity-im/profanity · PR #2050	52
Validating TIC input CSV structure nasa/ExoMiner · PR #2	53
Guarding the tab context menu against nulls phase1geo/Minder · PR #656	54
Preserving state before refresh initialization ultimatepp/ultimatepp · PR #293	55
Documenting timeout units in seconds psf/requests · PR #6994	56
Packaging ShaderExplorer assets for Android ezEngine/ezEngine · PR #1613	57
Removing duplicate build source entries cycfi/elements · PR #438	58
Adding a community contributor profile STICKnoLOGIC/First-Accord · PR #55	59
Resolving a Copilot panel-placement request microsoft/vscode-copilot-release · Issue #6998	60
Diagnosing workspace paths containing a plus sign coder/code-server · Issue #5483	61

[linebender/resvg](#) · #1135

Restoring inherited paint-server transforms in usvg

Pull request author · tests added by maintainer

Opened **2026-09-13** Merged **2026-10-03**

Merged upstream

The problem

In usvg's `paint_server.rs`, `convert_linear`, `convert_radial` and `convert_pattern` read their transform with `node.resolve_transform`, which ignores the href template chain. A template's `gradientTransform` or `patternTransform` was therefore dropped, as diagnosed in #1134. [PR record](#) · [Issue report](#)

Why it was needed

According to the report, the regression had shipped since v0.37.0 and remained in v0.48.1. Any fix had to restore inheritance without undoing the transform-origin support whose introduction caused the loss. [PR record](#) · [Issue report](#)

My contribution and the accepted change

My commit changes three call sites to `resolve_attr(node, ...).resolve_transform(...)`, so the element that defines the transform is found first and transform-origin is read from that same element. After luisbg said the fix looked good but asked for a regression test, LaurenzV added a commit with linear, radial and pattern via-href cases covering direct, inherited, chained and overridden use. [Changed files](#) · [Discussion](#) · [Commit](#)

Why this approach

Implementation-derived rationale: reusing the existing `resolve_attr` helper revives the `GradientTransform` entries already present in `resolve_lg_attr` and `resolve_rg_attr` instead of adding lookup code, and a transform on the element itself still takes precedence. The PR description documents both properties. [PR record](#) · [Changed files](#)

Validation and outcome

Author-reported: the reproducer passes, output matches `rsvg-convert` 2.58 within 1/255 per channel, and cargo test passed 1731 integration tests with no reference-image changes, on Linux only. My local paired tests were not submitted; the merged tests are LaurenzV's. LaurenzV squash-merged the PR on October 3, 2026. [PR record](#) · [Commit](#) · [Commit](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — killerdevildog, GitHub merge record, 2026-09-13[Accepted changed-file diff](#) — linebender/resvg, 2026-10-03[Squash merge commit](#) — LaurenzV, 2026-10-03[Review request for a regression test](#) — luisbg, 2026-09-14[Maintainer-added regression tests](#) — LaurenzV, 2026-10-03[Maintainer acceptance before merge](#) — LaurenzV, 2026-10-03[Linked bug report #1134](#) — Quaylyn Rimer / GitHub, 2026-09-13

[linebender/resvg](#) · #1134

Tracing lost gradient transforms to a regression

Bug reporter · regression analysis and merged fix

Opened **2026-09-13** Closed **2026-10-03****Resolved through my merged PR**

The problem

A gradient or pattern using another as a template through href inherited stops, coordinates, units and spreadMethod, but not gradientTransform or patternTransform. In my minimal SVG, a gold-to-navy gradient whose template is rotated 90 degrees rendered left to right in resvg instead of top to bottom. [Issue report](#)

Why it was needed

SVG 2 explicitly lists these transforms among the attributes copied from a template, and SVG 1.1 states the rule generally. The report showed the same loss for a radial gradient translation and a rotated pattern, so documents that reuse paint servers could render incorrectly. [Issue report](#)

My contribution and the resolution

I traced the cause to 9101f30a, the transform-origin change first released in v0.37.0: resolve_transform reads only the element itself, leaving the href-aware GradientTransform lookups unused. I proposed resolving the defining element first, attached before/after renders compared with rsvg-convert, and opened #1135, whose merge on October 3, 2026 closed the issue. [Issue report](#) · [Resolution](#)

Why this approach

Rather than patching the symptom, the report compared the v0.36.0 converters with the current ones and noted that no existing attributes-via-xlink:href test set a transform on the referenced element, explaining why the regression went unnoticed. The proposal keeps transform-origin support and lets a local transform win. [Issue report](#)

Validation and outcome

My pixel comparisons (within 1/255 after the fix, up to 255 before) are author-reported. Independently, matthargett commented that a new WPT retest covers this case, that usvg 0.48.1 renders it untransformed, and that Chromium 131 and Firefox agree on the expected output. LaurenzV closed the issue on 2026-10-03. [Issue report](#) · [Discussion](#) · [Resolution](#)

SOURCE RECORD

Original report · [linebender/resvg #1134](#) — Quaylyn Rimer / GitHub, 2026-09-13Resolving pull request #1135 by the reporter — [killerdevildog](#); GitHub merge record, 2026-10-03Independent WPT retest confirmation — [matthargett](#) / GitHub, 2026-10-03

ThePhysicsGuys/Physics3D · #28

Restoring the cylinder containment test

Pull request author · upstream contribution

Opened 2026-09-06 Merged 2026-09-21

Merged upstream

The problem

CylinderClass::containsPoint, the cylinder implementation behind Shape::containsPoint, evaluated its radial term as $\text{point.x} * \text{point.x} + \text{point.y} + \text{point.y}$. For cylinderShape(1, 2) the interior point (0, 0.75, 0) was rejected and the exterior point (0, -2, 0) was accepted. [PR record](#) · [Changed files](#)

Why it was needed

Containment should be symmetric about the cylinder's Z axis. Adding Y twice instead of squaring it made the answer depend on the sign of Y and accepted arbitrarily negative Y when X and Z were zero, so the error was not confined to boundary points. Companion report #29 documents the standalone reproducer. [PR record](#) · [Issue report](#)

My contribution and the accepted change

The accepted change is one line in Physics3D/geometry/builtinShapeClasses.cpp: $y + y$ becomes $y * y$, so the radial check reads $x*x + y*y \leq 1.0$ beside the unchanged $\text{abs}(z) \leq 1.0$ cap test. No other file, signature or shape class changed. The PR was opened about a minute before I filed #29, and its body links that issue with Fixes #29. [Changed files](#) · [Issue report](#)

Why this approach

Documented: the PR frames the defect as broken cylinder symmetry and restores the squared term. Implementation-derived: correcting the shared shape-class predicate fixes every containsPoint caller for cylinders at once, without touching the separate intersection-distance code below it in the same file. [PR record](#) · [Changed files](#)

Validation and outcome

Author-reported: a Debug build with GCC 16.0.1 passed all 13 geometry tests, and a local 30-assertion regression test over both signs of X and Y, two radii, and radial and cap boundaries failed before the fix and passed after. That test is quoted in the PR body but was not committed, and the full suite was not run. VonTum replied "Oh, you are correct. Well spotted." and merged the PR on September 21, 2026. [PR record](#) · [Commit](#) · [Discussion](#)

SOURCE RECORD

Pull request description and merge record — killerdevildog; GitHub merge record, 2026-09-06

Accepted changed-file diff — PR final diff, 2026-09-21

Merge commit — VonTum, 2026-09-21

Maintainer confirmation — VonTum, 2026-09-21

Companion bug report #29 — Quaylyn Rimer / GitHub, 2026-09-06

[ThePhysicsGuys/Physics3D](#) · #29

Reporting inverted cylinder containment

Bug reporter · standalone reproducer and root cause

Opened **2026-09-06** Closed **2026-09-21**

Resolved through my merged PR

The problem

`Shape::containsPoint` misclassified cylinders in both directions: for `cylinderShape(1, 2)`, the point `(0, 0.75, 0)` returned false and `(0, -2, 0)` returned true. I confirmed this on commit `00691df` with GCC 16.0.1, C++23 and a Debug build, using points away from the boundary. [Issue report](#)

Why it was needed

A containment query that rejects interior points and accepts exterior ones is wrong for ordinary inputs, not only edge cases. I traced it to `CylinderClass::containsPoint`, where `y + y` stood in for `y * y`, and explained that the expression breaks radial symmetry and accepts arbitrarily negative Y when X and Z are zero. [Issue report](#)

My contribution and the resolution

I supplied a short C++ program that links only the static physics library, the exact CMake and compiler commands, expected versus actual output, and a nonzero exit code as the pass/fail signal; no GUI or graphics dependencies are needed. On ordering: PR #28 was opened about a minute before this issue, which I filed as its companion report and pointed to #28 as the fix. [Issue report](#) · [Resolution](#)

Why this approach

Choosing an interior and an exterior point far from the boundary rules out tolerance arguments, and a self-contained program lets maintainers confirm the defect on unmodified source independently of the patch. That rationale is derived from the report's structure; the issue does not state it explicitly. [Issue report](#)

Validation and outcome

The report states that the wrong results reproduced on original source and that, after the fix, all 13 geometry tests passed, including an uncommitted 30-assertion regression test; it also says the full suite was not run. These results are author-reported. VonTum confirmed the bug and merged #28 on September 21, 2026, and this issue closed with that merge. [Issue report](#) · [Resolution](#) · [Discussion](#)

SOURCE RECORD

Original report · [ThePhysicsGuys/Physics3D #29](#) — Quaylyn Rimer / GitHub, 2026-09-06

My fix · [PR #28](#) — [killerdevildog](#); merged by VonTum, 2026-09-21

Maintainer confirmation on [PR #28](#) — VonTum / GitHub, 2026-09-21

FlaxEngine/FlaxEngine · #4262

Parsing plus-signed float exponents in Flax

Pull request author · upstream contribution

Opened 2026-09-13 Merged 2026-09-14

Merged upstream

The problem

ParseFloat folded exponent-sign handling into one ternary that advanced the cursor only for a minus. A plus sign stopped the digit loop immediately, and the trailing-character check returned failure before the result was stored, as reported in #4261. [PR record](#) · [Issue report](#)

Why it was needed

A plus-signed exponent is the form printf's %g produces, as the reproducer's reference column shows. Because many callers ignore the return value, a stale output was indistinguishable from success. Fixing the shared template repairs both the char and wide Char instantiations at once. [PR record](#) · [Issue report](#)

My contribution and the accepted change

Replaced the ternary with an explicit branch: a minus sets the multiplier to 0.1 and advances, a plus only advances, and the default stays 10. The accepted change is one commit with 10 additions and 1 deletion in StringUtilsBase.cpp. No test file was added. [Changed files](#)

Why this approach

Implementation-derived rationale: leaving powerer at 10 makes e+N behave exactly like eN, while input without a plus takes the same path as before. The PR description states this equivalence; no review comments or alternative designs appear in the record. [PR record](#) · [Changed files](#)

Validation and outcome

My harness extracted the before and after function bodies directly from git and compared them with strtod: 8/8 reported inputs fixed, 210 plus-free inputs, malformed ones included, bit-identical, and the wide Char path clean. These are author-reported results. Labeled core and fix under milestone 1.13, the PR was merged on September 14, 2026 by the flaxtechnology account; the merge commit is authored by Wojtek Figat. [PR record](#) · [Commit](#) · [Commit](#)

SOURCE RECORD

[Pull request description, verification harness and merge record](#) — killerdevidog; GitHub merge record, 2026-09-13

[Accepted changed-file diff](#) — FlaxEngine/FlaxEngine, 2026-09-14

[Merge commit](#) — Wojtek Figat, 2026-09-14

[Fix commit](#) — killerdevidog, 2026-09-13

[Linked bug report #4261](#) — Quaylyn Rimer / GitHub, 2026-09-13

Is-Daouda/is-Engine · #10

Keeping sprite offset setters independent

Pull request author · upstream contribution

Opened 2026-09-12 Merged 2026-09-14

Merged upstream

The problem

In is::Engine 4.0.x, MainObject::setXOffset() forwarded m_xOffset as the y argument to setXYOffset(). Because that helper assigns both members and calls updateSDMsprite(), changing the X offset also replaced the Y offset with the previous X value and moved the sprite, as reported in #9. [PR record](#) · [Issue report](#)

Why it was needed

Offsets position an object's sprite, and the header documents each setter as affecting one axis. Implementation-derived: until fixed, a caller wanting to change only X had to avoid setXOffset() and pass both values to setXYOffset() itself. [PR record](#) · [Changed files](#)

My contribution and the accepted change

The accepted patch changes one argument in app/src/main/cpp/MainObject.cpp: setXYOffset(val, m_xOffset) becomes setXYOffset(val, m_yOffset). The commit message explains the overwrite and carries Fixes #9. No header, public API or other setter was modified. [Changed files](#) · [Commit](#)

Why this approach

The corrected call mirrors setYOffset(), which already passed the untouched member for the other axis, so both setters share one pattern and still route through setXYOffset() and its sprite update. The PR explains that no test was added because the repository vendors catch.hpp but has no wired-up test suite or build target, and offers to upstream the harness. [PR record](#) · [Changed files](#)

Validation and outcome

Author-reported: the translation unit compiled cleanly on the SFML backend with GCC and -std=c++17, and a local harness against the real class showed Y staying at 20 after the change instead of dropping to 10. The mirrored setYOffset() case passed before and after. No reviews or comments appear; Is-Daouda merged the PR into 4.0.x on September 14, 2026. [PR record](#) · [Commit](#)

SOURCE RECORD

Pull request description and merge record — killerdevildog; GitHub merge record, 2026-09-12

Accepted changed-file diff — Changes reviewed at PR head, 2026-09-14

Squash-merge commit — Is-Daouda, 2026-09-14

Bug report · Is-Daouda/is-Engine #9 — Quaylyn Rimer / GitHub, 2026-09-12

FlaxEngine/FlaxEngine · #4261

Exposing silent float-parse failures on 1e+10

Bug reporter · reproducer and merged fix

Opened 2026-09-13 Closed 2026-09-14

Resolved through my merged PR

The problem

ParseFloat in StringUtilsBase.cpp consumed a minus sign after an exponent marker but never a plus. Inputs such as 1e+10, 6.02e+23 and 2.0E+8 left the cursor on the sign, so the function returned its failure code without ever writing the output value. [Issue report](#)

Why it was needed

The report traced ParseFloat behind both StringUtils::Parse float overloads and named callers that discard the result, including Variant::Parse, debug-console arguments and ProjectInfo XML values, so failures were silent. It also linked the gap to editor issue #197 and to the 2025 commit that added the exponent block without plus support. [Issue report](#)

My contribution and the resolution

I filed a self-contained reproducer that copies the function body verbatim, stubs only Math::Pow, and compares nine inputs against strtod, using a sentinel to prove the output stays unwritten. Fifteen minutes later I opened #4262 with the fix. A maintainer labeled the issue bug and core under the 1.13 milestone, and it closed when my PR merged on September 14, 2026. [Issue report](#) · [Resolution](#)

Why this approach

The diagnosis isolated the observable contract: unsigned and negative exponents worked, every plus-signed exponent failed, and callers kept stale values. Building the reproducer from the verbatim source made the defect checkable without compiling the engine. This rationale is drawn from the report's structure, not from maintainer comments. [Issue report](#)

Validation and outcome

The reproducer output and call-site analysis are author-reported; no maintainer comment appears on the issue. The code change and its before/after harness live in the resolving PR, which added no regression test. The issue was closed as completed on 2026-09-14. [Issue report](#) · [Resolution](#)

SOURCE RECORD

Original report · FlaxEngine/FlaxEngine #4261 — Quaylyn Rimer / GitHub, 2026-09-13

Resolving pull request #4262 by the reporter — killerdevildog; GitHub merge record, 2026-09-14

Is-Daouda/is-Engine · #9

Reporting an X setter that moved sprites vertically

Bug reporter · reproducer and follow-up PR

Opened **2026-09-12** Closed **2026-09-14****Resolved through my merged PR**

The problem

In is::Engine 4.0.3, calling `MainObject::setXOffset()` silently replaced the object's Y offset with its previous X offset. Because the shared helper also refreshes the sprite, the visible symptom was a sprite shifting on the Y axis when only its X offset was changed. [Issue report](#)

Why it was needed

The header documents the two setters as independent (Set the x offset / Set the y offset), and `setYOffset()` already honoured that, so the behaviour contradicted the documented contract rather than reflecting a design choice. Users had no signal beyond a misplaced sprite. [Issue report](#)

My contribution and the resolution

I quoted `setXOffset()`, `setYOffset()` and `setXYOffset()` side by side, showing the old X value passed as the y argument, and gave a short reproducer: `setXYOffset(10, 20)` then `setXOffset(5)` leaves Y at 10 instead of 20. I named the version, branch and commit (4.0.3, 4.0.x, 46e75c8). The issue was resolved by my own PR #10.

[Issue report](#) · [Pull request](#)

Why this approach

Placing the faulty setter next to its correct mirror let a maintainer confirm the bug by reading alone, without building the engine. The report separated expected from actual behaviour and noted that, although reproduced on SFML, the code path is backend-independent. [Issue report](#)

Validation and outcome

Reproduction on the SFML backend under Linux, GCC and C++17 is author-reported. The report already proposed the one-line `m_xOffset` to `m_yOffset` fix and announced a PR. No maintainer comment appears on the issue. It was closed on September 14, 2026, when Is-Daouda merged the linked PR, and the squash commit carries Fixes #9. [Issue report](#) · [Pull request](#) · [Resolution](#)

SOURCE RECORD

[Original report · Is-Daouda/is-Engine #9](#) — Quaylyn Rimer / GitHub, 2026-09-12[Author's fix · Is-Daouda/is-Engine #10](#) — killerdevildog / GitHub, 2026-09-12[Squash-merge commit closing the issue](#) — Is-Daouda, 2026-09-14

[x-ege/xege](#) · #399

Returning full-range angles between 3D vectors

Pull request author · upstream contribution

Opened **2026-09-12** Merged **2026-09-13**

Merged upstream

The problem

As diagnosed in #398, GetAngle computed only the cross-product magnitude of the normalized inputs and returned its asin, whose range stops at $\pi/2$. Obtuse inputs were reflected, and +x against -x reported 0 degrees. [PR record](#) · [Issue report](#)

Why it was needed

The PR's before/after table shows 120, 135 and 180 degree cases, plus the documented default argument, returning wrong values on master. Half of all angles between 0 and 180 degrees were affected, so callers received wrong results whenever two vectors were more than 90 degrees apart. [PR record](#)

My contribution and the accepted change

The two-line change in src/math.cpp computes $cr = s * e$ beside sr and returns $\text{atan2}(sr, cr)$ instead of $\text{asin}(sr)$. No include was added, since atan2 comes from the math.h already included. No test file was added. The change landed on master as one commit authored by me and committed by Ltabsyy. [Changed files](#) · [Commit](#)

Why this approach

Documented rationale: atan2 of the sine and cosine components is well defined across the full 0 to π range, both values were already available in the function, and the sibling Rotate overload already pairs cr with sr , so the change brings the two into agreement. [PR record](#)

Validation and outcome

Author-reported: a local harness of hand-computed cases, two-plane sweeps, and symmetry and scale checks went from 275 failures to 0; ctest passed 14/14; ASan/UBSan Debug runs were clean. The harness was not submitted. CodeRabbit's automated review raised no actionable comments. Ltabsyy wrote "I think it's okay. Shall we merge first?" and merged on September 13, 2026. [PR record](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

Pull request description and merge record — [killerdevildog](#); GitHub merge record, 2026-09-12

Accepted changed-file diff — [x-ege/xege](#), 2026-09-13

Merged commit on master — [killerdevildog](#); committed by Ltabsyy, 2026-09-13

Automated CodeRabbit review summary — [coderabbitai\[bot\]](#), 2026-09-12

Linked bug report #398 — [Quaylyn Rimer](#) / GitHub, 2026-09-12

Maintainer merge comment — Ltabsyy, 2026-09-13

LibreSprite/LibreSprite · #649

Restoring true sprite height in crash recovery

Pull request author · upstream contribution

Opened 2026-09-07 Merged 2026-09-13

Merged upstream

The problem

As diagnosed in #648, the DocumentInfo-only branch of Reader::readSprite() assigned the backup's width to DocumentInfo::height, discarding the height it had just range-checked. Crash recovery therefore treated every non-square sprite as W x W. [PR record](#) · [Issue report](#)

Why it was needed

Two consumers read those fields: the Recover Files label built in session.cpp, and the canvas that read_document_with_raw_images() creates for Raw Frames and Raw Layers recovery. Without the fix, raw recovery rebuilt artwork on a canvas of the wrong size, not just under a wrong label. [PR record](#)

My contribution and the accepted change

The merged patch changes one line in src/app/crash/read_document.cpp, from m_loadInfo->height = w to m_loadInfo->height = h, with a commit message tracing both consumers and closing #648. The backup format and the rest of the recovery flow are unchanged. [Changed files](#) · [Commit](#)

Why this approach

The PR grounds the choice in LibreSprite's own code: the normal path a few lines below already builds new Sprite(format, w, h, 256). It also documents provenance: written against local code rather than cherry-picked from Aseprite's matching 2017 fix, which the PR notes post-dates the 2016 relicense and is not GPL-compatible with this project. [PR record](#) · [Changed files](#)

Validation and outcome

No full CMake build ran (the tree needs CMake 4.1; the author had 3.28), and the project test suite was not run. Instead, an author-reported harness linked the unmodified reader against base, gfx, doc and undo and showed 320x320 before and 320x200 after. No tests were added. Nidrax approved on September 7; felipemanga approved and merged on September 13, 2026. [PR record](#) · [Commit](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — killerdevildog; GitHub merge record, 2026-09-07

[Accepted changed-file diff](#) — Changes reviewed at PR head, 2026-09-13

[Merged commit on master](#) — felipemanga, 2026-09-13

[Approval by Nidrax](#) — Nidrax, 2026-09-07

[Approval by felipemanga](#) — felipemanga, 2026-09-13

[Bug report](#) · [LibreSprite/LibreSprite #648](#) — Quaylynn Rimer / GitHub, 2026-09-07

x-ege/xege · #398

Diagnosing supplement angles in a vector helper

Bug reporter · root cause and merged fix

Opened 2026-09-12 Closed 2026-09-13

Resolved through my merged PR

The problem

VECTOR3D::GetAngle returned asin of the cross-product magnitude. Because sine is symmetric about 90 degrees, every obtuse angle came back as its supplement and antiparallel vectors reported 0, although ege.h documents the function simply as the angle between two vectors. [Issue report](#)

Why it was needed

The report swept every degree from 0 to 180 and found 90 of 181 results wrong, including the documented default +z argument, where an input of -z returned 0 instead of pi. The fault is in shared arithmetic, so it is not specific to any backend or platform. [Issue report](#)

My contribution and the resolution

I filed the report against commit 4148e1f with a Chinese summary, a five-case reproducer, its actual output and the faulty lines in src/math.cpp, then proposed atan2 over the cross magnitude and dot product. Within two minutes I opened #399 with that change; Ltabsy merged it on September 13, 2026, closing the issue. [Issue report](#) · [Resolution](#)

Why this approach

The diagnosis named the missing information, the sign of the dot product, instead of patching individual cases. It also pointed out that the sibling Rotate overload directly above already computes the dot product beside the cross magnitude, giving the maintainers an in-codebase precedent for the fix. [Issue report](#)

Validation and outcome

The reproducer output, the 362/362 two-plane sweep and the 14/14 ctest result are author-reported. No maintainer discussion appears on the issue itself; the code change and fuller validation are in #399. Ltabsy closed the issue as completed on 2026-09-13. [Issue report](#) · [Resolution](#)

SOURCE RECORD

Original report · x-ege/xege #398 — Quaylyn Rimer / GitHub, 2026-09-12

Resolving pull request #399 by the reporter — killerdevildog; GitHub merge record, 2026-09-13

LibreSprite/LibreSprite · #648

Tracing square-canvas crash recovery to its source

Bug reporter · harness, root cause and follow-up PR

Opened 2026-09-07 Closed 2026-09-13

Resolved through my merged PR

The problem

LibreSprite's crash recovery reported the wrong height for every backup: a 320x200 sprite appeared in Recover Files as RGB Sprite 320x320, and the Raw Frames and Raw Layers options restored it onto a 320x320 canvas. The cause was one line in `read_document.cpp` storing the width as the height. [Issue report](#)

Why it was needed

Recovery runs after a crash, when a user is trying to get lost work back. The report traced both consumers of the bad field by file and line, showing that it changed recovered output, not merely a label, for any non-square sprite. [Issue report](#)

My contribution and the resolution

I described a harness that compiles `read_document.cpp` unmodified, writes a 320x200 backup in `write_document()`'s on-disk format and calls the real `read_document_info()`, with output before and after a one-token change. I traced the slip to a 2015 commit and noted Aseprite fixed the same line in 2017, after LibreSprite forked from v1.1.7. It was resolved by my own PR #649. [Issue report](#) · [Pull request](#)

Why this approach

Driving the unmodified reader without GUI, SDL or CMake isolated the fault to a single assignment. The history check supported intent: `h` is validated and then unused, and the non-info path already builds `Sprite(format, w, h, 256)`. A possible link to #509 was explicitly marked unverified. [Issue report](#)

Validation and outcome

The harness output is author-reported. Although the report says the harness is attached, no harness code or comment appears in the retrieved issue record. [felipemanga](#) closed the issue on September 13, 2026 with merged commit 0630070, which retains my authorship. [Issue report](#) · [Pull request](#) · [Resolution](#)

SOURCE RECORD

Original report · LibreSprite/LibreSprite #648 — Quaylyn Rimer / GitHub, 2026-09-07

Author's fix · LibreSprite/LibreSprite #649 — killerdevidog / GitHub, 2026-09-07

Merged commit closing the issue — felipemanga, 2026-09-13

raysan5/raylib · #6144

Correcting SHA-256 padding for edge-case lengths

Pull request author · upstream contribution

Opened **2026-09-12** Merged **2026-09-12**

Merged upstream

The problem

Issue #6143 showed `ComputeSHA256()` returning wrong digests whenever the message length was 56 to 59 mod 64. The padding reserve used `sizeof(dataSize)`, 4 bytes for an int, while the code below wrote a 1-byte marker and an 8-byte length, so in those cases the length overwrote the marker. [PR record](#) · [Issue report](#)

Why it was needed

A hash that disagrees with FIPS 180-4 for one input length in sixteen cannot be compared with digests from other tools. The PR also noted that in the failing cases the padded buffer was one block short of the canonical size the standard requires. [PR record](#)

My contribution and the accepted change

The merged change is one line in `src/rcore.c`: the reserve becomes `sizeof(bitLen)`, with a comment saying it covers the 64-bit length appended at the end. The existing round-up expression was left unchanged. The PR states that digests for the affected lengths intentionally change, because they now match the standard. [Changed files](#) · [Commit](#)

Why this approach

Documented in the PR: with an 8-byte reserve, the existing round-up yields exactly the canonical size. Writing $r = (\text{dataSize} + 8) \bmod 64$, room after the message is $8 + (64 - r)$, at least 9 bytes, and $r = 0$ adds the extra block the standard requires. Keying the reserve to `bitLen` also matches the write loop, which already used `sizeof(bitLen)`. [PR record](#) · [Changed files](#)

Validation and outcome

Author-reported: all five NIST vectors pass, every length from 0 to 4,096 and 3,000 random messages match a reference SHA-256, padding shows 0 mismatches over 10,000,001 lengths, and ASan/UBSan runs are clean. Testing was Linux/GCC only and the harnesses stayed local, so no tests were added. raysan5 merged the PR on September 12, 2026, about three hours after it opened, without a formal review, and posted a thank-you comment. [PR record](#) · [Commit](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — killerdevildog; GitHub merge record, 2026-09-12

[Accepted changed-file diff](#) — Changes reviewed at PR head, 2026-09-12

[Squash-merge commit](#) — raysan5, 2026-09-12

[Maintainer comment after merge](#) — raysan5, 2026-09-12

[Bug report](#) · raysan5/raylib #6143 — Quaylyn Rimer / GitHub, 2026-09-12

[raysan5/raylib](#) · #6143

Diagnosing silently wrong SHA-256 digests

Bug reporter · reproducer and follow-up PR

Opened **2026-09-12** Closed **2026-09-12****Resolved through my merged PR**

The problem

ComputeSHA256() in raylib's rcore.c returned an incorrect hash whenever the input length was 56, 57, 58 or 59 mod 64, including the published 56-byte FIPS 180-4 two-block test vector. Every write stayed inside the allocation, so the failure produced a plausible digest instead of a crash. [Issue report](#)

Why it was needed

The defect was deterministic rather than data dependent: within those four residue classes every length failed, which is 6.25% of all input lengths. With no crash or out-of-bounds access, nothing short of comparison against a reference implementation would reveal it. [Issue report](#)

My contribution and the resolution

I filed the report against master at 2f768eb with a minimal program printing raylib's digest for the FIPS vector beside the expected value, which sha256sum confirmed. I traced the cause to a 4-byte reserve, sizeof(dataSize) on an int, versus the 9 bytes of marker and 64-bit length written later. The issue was resolved by my own PR #6144. [Issue report](#) · [Pull request](#)

Why this approach

Sweeping lengths turned a suspected typo into a bounded defect: 256 wrong of 4,097 lengths, 210 of 3,000 random messages, and 625,000 padding mismatches up to 10,000,000, all in the same four classes. Checking SHA-1, MD5, CRC32 and Base64 against references confirmed the scope; SHA-1 already reserved the full 8 bytes. [Issue report](#)

Validation and outcome

All measurements are author-reported from local harnesses that were not published; they were not re-run for this portfolio. The issue received no maintainer comment. raysan5 closed it on September 12, 2026, about three hours after filing, when the linked PR merged. [Issue report](#) · [Pull request](#) · [Resolution](#)

SOURCE RECORD

Original report · [raysan5/raylib #6143](#) — Quaylyn Rimer / GitHub, 2026-09-12

Author's fix · [raysan5/raylib #6144](#) — killerdevildog / GitHub, 2026-09-12

Squash-merge commit closing the issue — raysan5, 2026-09-12

[opentoonz/opentoonz](#) · [#7113](#)

Tokenizing scientific notation in expressions

Pull request author · upstream contribution

Opened **2026-09-06**Merged **2026-09-10**

Merged upstream

The problem

`TSyntax::Tokenizer::setBuffer()` scanned exponents only inside its decimal-point branch and always required `s[i + 2]` to be a digit. Literals such as `1.0e2`, `1.0e0`, `1e2` and `1e+2` were split into a `Number` plus further tokens, while `1.0e+2` and `1.0e02` worked. [PR record](#) · [Changed files](#) · [Issue report](#)

Why it was needed

The expression parser converts `Number` tokens into values, so a split literal is broken before evaluation, and the expression field uses the same parser for syntax highlighting. Report #7112 showed 5 of 10 inputs failing against the unchanged tokenizer. [Issue report](#)

My contribution and the accepted change

The accepted change touches only `tokenizer.cpp` (+9/-7). Exponent scanning moves after the optional fraction, so it applies with or without a decimal point. After `e` or `E`, an optional sign is skipped and the exponent is consumed only if a digit follows; otherwise the `e` is left for the identifier scanner. A token `!= "."` guard stops a bare point acting as the mantissa. [PR record](#) · [Changed files](#)

Why this approach

Documented: independent exponent scanning, an optional sign and a mandatory digit, as the report suggested. From the code: looking ahead before appending keeps malformed forms such as `1.0e+` from swallowing characters. RodneyBaker called the patch correct, minimal and well-scoped and suggested a `bool` instead of the `"."` comparison, though not as a blocker; the merged diff keeps the comparison. [PR record](#) · [Changed files](#) · [Discussion](#)

Validation and outcome

Author-reported: the 10-case reproducer passes; a local 34-case harness passes and fails on the original source; ASan, UBSan and LSan are clean. RodneyBaker posted a checklist (malformed exponents, `1e2+3`, identifiers, `.5`, evaluated values) and I reported every check passing locally. No tests were committed and the GUI was not tested. RodneyBaker merged it on September 10, 2026. [PR record](#) · [Commit](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — [killerdevildog](#); GitHub merge record, 2026-09-06[Accepted changed-file diff](#) — PR final diff, 2026-09-10[Merge commit](#) — RodneyBaker, 2026-09-10[Maintainer review and test checklist](#) — RodneyBaker, 2026-09-08[Linked bug report #7112](#) — Quaylyn Rimer / GitHub, 2026-09-06

[opentoonz/opentoonz](#) · #7112

Reporting split exponent literals

Bug reporter · reproducer and root-cause analysis

Opened 2026-09-06 Closed 2026-09-10

Resolved through my merged PR

The problem

The OpenToonz expression tokenizer split `1.0e2` into a `Number 1.0` and an `Ident e2`, while `1.0e+2` and `1.0e02` each produced one token. Uppercase `E`, exponent zero, `1e2` and `1e+2` failed as well. I reproduced this against upstream commit `868d795` with `GCC 16.0.1` and `Qt5Core 5.15.13`. [Issue report](#)

Why it was needed

Token output decides the literal that `NumberPattern::createNode()` converts, and the same parser drives syntax highlighting. In the linked lines I identified two causes: the exponent test always demanded a digit at `s[i + 2]`, and exponent scanning was nested inside the decimal-point branch. [Issue report](#)

My contribution and the resolution

I provided a 10-case harness compiled only against `ttokenizer.cpp` and `Qt5Core`, the build command, and actual token streams with 5 of 10 failing. I proposed scanning the exponent after the fraction with an optional sign and a required digit, listed malformed inputs for regression coverage, and gave `1.0e+2` as a workaround. My PR #7113, opened seven minutes later, implemented it. [Issue report](#) · [Resolution](#)

Why this approach

Driving the production tokenizer in isolation needs no application build or GPU, as the report notes. The claims were also bounded: wrong token streams were confirmed, but the evaluator and GUI were explicitly not tested, so no displayed value or GUI error was asserted. The report states no fix had been applied or validated at filing time. [Issue report](#)

Validation and outcome

RodneyBaker labeled the issue a bug on September 9, 2026, and it closed on September 10 when he merged #7113. His pre-merge checklist covered the malformed exponents this report listed (`1.0e`, `1.0e+`, `1.0e-`) alongside evaluated values; my passing results for it are author-reported. [Issue report](#) · [Resolution](#) · [Discussion](#)

SOURCE RECORD

Original report · [opentoonz/opentoonz #7112](#) — Quaylyn Rimer / GitHub, 2026-09-06

My fix · [PR #7113](#) — [killerdevildog](#); merged by RodneyBaker, 2026-09-10

Maintainer pre-merge checklist — RodneyBaker / GitHub, 2026-09-08

[xissburg/edyn](#) · #156

Correcting a sign in polyhedron raycasts

Pull request author · upstream contribution

Opened 2026-09-06

Merged 2026-09-08

Merged upstream

The problem

shape_raycast for polyhedron_shape has a branch for segments parallel to a face. It computed $\text{dist} = \text{dot}(\text{vertex} - p0, \text{normal})$ and returned no hit when $\text{dist} > 0$, which with outward normals is the interior side of the plane. A segment through the center of a `make_box_mesh()` cube therefore missed. [PR record](#) · [Changed files](#)

Why it was needed

Any axis-aligned segment through a box mesh is parallel to four of its faces, so the bug hit ordinary queries. Report #155 showed box_shape returning fraction 0.25 while the same cube as a polyhedron returned EDYN_SCALAR_MAX, the no-hit value, on all three axes. [PR record](#) · [Issue report](#)

My contribution and the accepted change

The merged change flips one comparison in `src/edyn/collision/raycast.cpp` from $\text{dist} > 0$ to $\text{dist} < 0$, so only segments lying outside a parallel face's plane are rejected. The adjacent code comment already said the segment misses when it lies in front of the face; the condition now matches it. The non-parallel clipping path is untouched. [Changed files](#)

Why this approach

Documented: the PR derives the sign from the outward-normal convention, where negative dist places p0 outside the face. After review, xissburg confirmed the bug and found the same mistake in the Real-Time Collision Detection reference algorithm the code had been copied from. That a sign fix is the right scope, rather than a rewrite, is an inference from that origin. [PR record](#) · [Discussion](#)

Validation and outcome

Author-reported: the reproducer fails before and passes after; 30 local rays covering both directions on each axis, interior and boundary hits, misses, normals and face indices passed; the full suite passed 66 of 71, the same five failing on the unmodified baseline. The regression test was not committed. xissburg merged the PR on September 8, 2026. [PR record](#) · [Commit](#)

SOURCE RECORD

[Pull request description and merge record](#) — killerdevildog; GitHub merge record, 2026-09-06[Accepted changed-file diff](#) — PR final diff, 2026-09-08[Merge commit](#) — xissburg, 2026-09-08[Maintainer confirmation and root-cause origin](#) — xissburg, 2026-09-08[Linked bug report #155](#) — Quaylyn Rimer / GitHub, 2026-09-06

[xissburg/edyn](#) · #155

Diagnosing missed polyhedron ray hits

Bug reporter · reproducer and root-cause diagnosis

Opened 2026-09-06

Closed 2026-09-08

Resolved through my merged PR

The problem

A segment through a cube built with `make_box_mesh()` returned no hit when the cube was a `polyhedron_shape`, while the equivalent `box_shape` returned 0.25. I reproduced this on all three axes at Edyn 1.3.1 (commit ee086e1) with Clang 18.1.3 and EnTT 3.15.0, and checked the same condition on master. [Issue report](#)

Why it was needed

The failing segments pass straight through the center, so the defect reaches routine axis-aligned picking and ray queries on convex polyhedra, not just grazing cases. The polyhedron result was `EDYN_SCALAR_MAX`, the default no-hit fraction, so callers received a clean miss rather than an error. [Issue report](#)

My contribution and the resolution

I wrote a self-contained program comparing both shapes along each axis with a nonzero exit on failure, linked the parallel-face branch, and worked one case: for the +Y face, a ray along X has `dist = 1` and `denom = 0` and is wrongly rejected. I proposed the `dist < 0` diff and opened PR #156 seven minutes after the report; its merge closed this issue. [Issue report](#) · [Resolution](#)

Why this approach

Using `box_shape` as a known-good oracle on identical input isolates the polyhedron path; that framing is derived from the reproducer rather than stated in it. The report's explicit argument is the outward-normal sign convention. Maintainer xissburg reviewed the code, confirmed the bug, and traced it to the Real-Time Collision Detection reference algorithm, which contains the same mistake. [Issue report](#) · [Discussion](#)

Validation and outcome

The report states that changing only that comparison in a temporary source copy made all three results 0.25 and the reproducer exit successfully, and that suite failures matched the unmodified baseline. These are author-reported. PR #156 merged on September 8, 2026, closing the issue; xissburg then thanked me for the find. [Issue report](#) · [Resolution](#) · [Discussion](#)

SOURCE RECORD

Original report · [xissburg/edyn #155](#) — Quaylyn Rimer / GitHub, 2026-09-06My fix · [PR #156](#) — [killerdevildog](#); merged by xissburg, 2026-09-08Maintainer confirmation — [xissburg](#) / GitHub, 2026-09-08

DiligentGraphics/DiligentCore · #804

Correctly parsing scientific notation in shaders

Bug reporter · reproducer and proposed patch

Opened **2026-08-17** Closed **2026-09-06****Resolved by upstream maintainer**

The problem

SkipFloatNumber required an explicit plus or minus sign after an exponent marker. Valid literals such as 1e10 and 6.02e23 were consumed only up to the exponent. HLSL tokenization consequently split a single number into a numeric constant and a spurious identifier. [Issue report](#)

Why it was needed

The same helper serves number-length counting and shader tokenization. Misclassification could bypass the HLSL-to-GLSL converter's numeric-suffix handling and expose later identifier transformations to a token that should never have existed. My report contrasted failing unsigned exponents with working signed forms. [Issue report](#)

My contribution and the resolution

I supplied a minimal C++ reproducer, a table of expected versus actual results, a tokenizer example, and a proposed patch. The maintainer's closing commit adopted the optional-sign logic and added regression coverage in both ParsingToolsTest.cpp and StringToolsTest.cpp. The upstream implementation is credited to the maintainer, not to a PR authored by me. [Issue report](#) · [Resolution](#)

Why this approach

An optional sign is consumed only when present; the existing digit check still rejects incomplete or malformed exponents. This fixes the grammar at the shared parser instead of adding special cases to individual shader consumers. That rationale and the exact proposed change are documented in the report. [Issue report](#) · [Resolution](#)

Validation and outcome

I reported locally checking the listed failures and preserving 94 existing number-counting assertions. Those are author-reported results, not a new benchmark run for this portfolio. The inspected upstream commit adds unsigned-exponent counting and tokenizer assertions and closes #804. [Issue report](#) · [Resolution](#)

SOURCE RECORD

Original report · DiligentGraphics/DiligentCore #804 — Quaylyn Rimer / GitHub, 2026-08-17

Maintainer fix and regression tests — Diligent Graphics, 2026-09-06

[turanszkij/WickedEngine](#) · #1695

Recovering sphere hits when a ray starts inside

Bug reporter · compiled reproducer and proposed fix

Opened **2026-08-22** Closed **2026-08-23****Resolved by upstream maintainer**

The problem

`Sphere::intersects` calculated the far-root hit, stored its distance and normal, then fell through to return false. A ray beginning inside a sphere could therefore return a miss even while the output distance described the correct exit point. [Issue report](#)

Why it was needed

The report traced this helper into sphere and capsule collider raycasts, ragdoll picking, editor picking, and Lua bindings. It supplied cases for inside origins and a TMin that skips the entry root. The defect affected control flow after a valid geometric calculation, not the calculation alone. [Issue report](#)

My contribution and the resolution

I linked the affected lines, explained the two quadratic roots, supplied a reproducer compiled against engine code, and proposed the missing return true. The maintainer's closing commit adds that return and also accepts zero discriminants and inclusive TMin/TMax boundaries. Those extra boundary changes are the maintainer's implementation. [Issue report](#) · [Resolution](#)

Why this approach

Returning success from the far-root branch makes it consistent with the already-working near-root branch and preserves the computed distance and normal. This is a targeted correction at the shared primitive, so downstream callers benefit without separate workarounds. The report explicitly motivates the return; the broader boundary handling appears in the maintainer diff. [Issue report](#) · [Resolution](#)

Validation and outcome

The report includes observed distances, several affected overloads, a capsule case, and genuine-miss controls. Its local verification is author-reported. The maintainer replied that the issue was fixed and committed the changes on August 23, 2026; the issue was then closed. [Issue report](#) · [Resolution](#) · [Discussion](#)

SOURCE RECORD

[Original report](#) · [turanszkij/WickedEngine #1695](#) — Quaylyn Rimer / GitHub, 2026-08-22[Maintainer geometry fix](#) — Turánszki János, 2026-08-23[Maintainer resolution confirmation](#) — turanszkij / GitHub, 2026-08-23

OpenLoco/OpenLoco · #3614

Making loaded-object iteration easier

Pull request author · upstream contribution

Opened 2026-01-13

Merged 2026-07-23

Merged upstream

The problem

Code iterating loaded OpenLoco objects repeatedly scanned all possible IDs, fetched each object and skipped null slots. This duplicated iteration mechanics at call sites such as map color assignment. [PR record](#)

Why it was needed

A reusable traversal helper makes the intent of object processing clearer and centralizes the loaded-object check. Some callers need only the object, while others also need its stable loaded ID for indexed arrays such as industry colors. [PR record](#)

My contribution and the accepted change

Added templated `ObjectManager::forEachLoaded<T>` with compile-time callback selection using `std::is_invocable_v`. It loops over valid ID capacity, obtains each object and invokes the callback only when non-null. The documented final signatures provide const object references, with an optional `LoadedObjectId`. Four manual land/industry loops in `MapWindow` were converted to callbacks, and `CONTRIBUTORS.md` gained `killerdevildog`. [Changed files](#)

Why this approach

Documented: reduce boilerplate and improve readability while supporting ID-aware and object-only usage. The original proposal included mutable callbacks; a reviewer explained that loaded objects should not be mutable outside loading/unloading, so the followup removed the mutable variants from the advertised API. This review changed the final design, rather than merely approving the initial PR description. [PR record](#) · [Changed files](#) ·

[Discussion](#) · [Discussion](#)

Validation and outcome

The GitHub record records approvals on 2026-04-24 and 2026-07-23, with merge by LeftofZen on July 23. Another contributor noted that they had looked for such a helper while working elsewhere. The final patch demonstrates the helper on existing map logic; no new test file or test execution log appears in the examined material. Merged 2026-07-23 by LeftofZen into OpenLoco/OpenLoco:master. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

Pull request description and merge record — [killerdevildog](#); GitHub merge record, 2026-01-13

Accepted changed-file diff — [Changes reviewed at PR head](#), 2026-07-23

Accepted changed-file diff — [Changes reviewed at PR head](#), 2026-07-23

Review rationale for removing mutable variants — [duncanspumpkin](#), 2026-01-14

Remove mutable callback variants — [Changes reviewed at PR head](#), 2026-01-14

Final approval — [LeftofZen](#), 2026-07-23

[killerdevildog/checkmate · #1](#)

Integrating role-specific requirements into a proposal

Author and maintainer · personal documentation project

Opened **2026-05-12** Closed **2026-05-14****Completed in personal repository**

The problem

The issue proposed organizing accountability provisions for publicly appointed defense counsel and prosecutors, separating role-specific duties from shared conduct. It identified a scope boundary around delegated public authority. This is a personal policy-drafting repository, not an upstream software product or enacted law. [Issue report](#)

Why it was needed

The proposal needed a consistent home for related provisions, examples, classifications, and cross-references. The issue's organization distinguished public-defender provisions, prosecutor provisions, and shared provisions so the intended responsibilities would not be treated as interchangeable. These are the author's proposed requirements. [Issue report](#)

My contribution and the resolution

I authored the issue and documented its incorporation into the Due Process section. The closing comment points to Sections 2.6.2, 2.11, and 2.12. The referenced follow-up commit expands public-defender duties and workload-related provisions in the source section and assembled proposal, explicitly referring to issue #1. [Issue report](#) · [Discussion](#) · [Discussion](#)

Why this approach

Grouping material by role and keeping shared provisions separate makes the document's structure easier to inspect and maintain. Using a dedicated source section plus an assembled proposal keeps the text available in both modular and complete forms. The structural reasoning is inferred from the files and issue organization, not a claim of legal validation. [Issue report](#) · [Discussion](#)

Validation and outcome

The issue was closed by me on May 14, 2026. The closing comment names release v5.8.7; the inspected commits support continued document integration and cross-reference work. This is credited as personal requirements/documentation work, with no claim that another organization accepted code or that any proposed provision became law. [Discussion](#) · [Discussion](#)

SOURCE RECORD

[Original report · killerdevildog/checkmate #1](#) — Quaylyn Rimer / GitHub, 2026-05-12[Author's integration summary](#) — Quaylyn Rimer / GitHub, 2026-05-14[Issue-linked documentation commit](#) — Quaylyn Rimer / checkmate, 2026-05-14

tinygs/tinyGS · #306

Selecting the correct SX1280 radio driver

Pull request author · upstream contribution

Opened 2026-03-17

Merged 2026-03-17

Merged upstream

The problem

Radio::init() selected the SX1280 radio implementation, but the switch case lacked a break. Execution immediately continued into the default SX1268 branch, replacing the radioHal pointer and module name after constructing the intended SX1280 object. [PR record](#)

Why it was needed

The PR explains that boards configured for SX1280, including the LilyGo T3S3 2.4GHz example, could therefore initialize the wrong hardware driver. The overwritten pointer also left the first allocated object unreachable. This was a control-flow defect at hardware initialization, where preserving the selected radio type is essential. [PR record](#)

My contribution and the accepted change

Added a single break statement after moduleNameString="SX1280" in tinyGS/src/Radio/Radio.cpp. The final diff changes one line and leaves the driver construction arguments and default driver selection intact. The direct code effect is to end the SX1280 branch before default can overwrite its state. [Changed files](#)

Why this approach

Documented: the author explicitly chose the same break structure already used by the other switch cases. This narrowly restores the intended one-driver selection without restructuring RadioHal ownership or adding another platform abstraction. The implementation supports the wrong-driver diagnosis directly; broader operational consequences are explained in the PR rather than demonstrated by hardware logs. [PR record](#) · [Changed files](#) ·

[Discussion](#)

Validation and outcome

G4lile0 thanked the contributor and merged the change into beta on 2026-03-17. No automated tests, hardware run log or measured field outcome are present in the examined PR. Acceptance and the one-line control-flow correction are verified; a guarantee that all affected stations worked afterward would exceed the evidence.

Merged 2026-03-17 by G4lile0 into tinygs/tinyGS:beta. [PR record](#) · [Changed files](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — killerdevildog; GitHub merge record, 2026-03-17

[Accepted changed-file diff](#) — Changes reviewed at PR head, 2026-03-17

[G4lile0 acknowledges contribution](#) — G4lile0, 2026-03-17

[Author diagnosis in commit message](#) — Changes reviewed at PR head, 2026-03-17

libuv/libuv · #4843

Preserving const correctness in Linux code

Pull request author · upstream contribution

Opened 2025-07-19 Merged 2026-02-21

Merged upstream

The problem

uv_cpu_info in src/unix/linux.c used a mutable char pointer for data including the string literal unknown. The author linked issue #4814 and described warnings under strict const-correctness options such as -Wwrite-strings. [PR record](#)

Why it was needed

The relevant pointer is used for reading CPU information. Matching its type to that read-only use removes a const mismatch at the declaration, helping strict builds without changing the CPU-information algorithm or introducing extra copies. [PR record](#)

My contribution and the accepted change

The final accepted patch changes one local declaration from char* p to const char* p. It does not add the explicit cast described by the original PR body. The scope is one declaration in one source file, and no runtime behavior, public signature, data structure, or platform implementation was otherwise changed. [Changed files](#)

Why this approach

Documented review rationale: vtjnash asked whether p could instead be marked const char*. In a February 21, 2026 reply, killerdevildog confirmed updating the declaration and called the suggestion cleaner. This preserves the compiler's read-only type information instead of suppressing the warning with a cast. The final diff supersedes the cast proposed in the original description. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The GitHub record contains no added test file or benchmark. The author described the warning and corrected the code after review; that supports a narrow compile-warning claim, not a separately verified full strict-build matrix. Read-only semantics are visible in the final patch and review explanation. vtjnash approved the earlier review with the const suggestion; the PR was merged February 21, 2026. The final merged change is the const declaration. This case illustrates following maintainer guidance on a minimal correctness cleanup. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

Fix const-correctness warning in linux.c — [pull request and author description](#) — killerdevildog, 2025-07-19

Accepted implementation — [changed files](#) — libuv/libuv, 2026-02-21

Merge commit — vtjnash, 2026-02-21

Review discussion by vtjnash — vtjnash, 2025-09-25

Review discussion by killerdevildog — killerdevildog, 2026-02-21

killerdevildog/GDevelop · #1

Exploring spatial hashing for collision detection

Personal fork · experiment author

Opened 2026-02-17 Merged 2026-02-17

Self-merged in personal fork · no upstream acceptance

The problem

The author identified all-pairs collision candidate enumeration in `hitBoxesCollisionTest`: every object in one group was considered against the other group, giving $O(N \times M)$ pair work before geometric testing. [PR record](#)

Why it was needed

The fork PR reports that a 1,000-object benchmark took about 43 ms in the original path, exceeding a 60 FPS frame interval. Its objective was to reduce expensive collision tests in crowded scenes by excluding objects in distant regions before the existing narrow-phase check. [PR record](#)

My contribution and the accepted change

Added a generic TypeScript `SpatialHashGrid` mapping object axis-aligned bounds into numeric-hashed cells. The collision routine builds it from the second object list and queries nearby candidates from the first, retaining `RuntimeObject.collisionTest` for actual intersections. Reused grid storage and pooled arrays limit repeated allocation. Below 32 total objects, the previous `twoListsTest` path remains. Cell size is twice the second list's average maximum dimension, with a 32-pixel minimum. [Changed files](#)

Why this approach

Documented: the threshold avoids grid-building overhead for small groups; adaptive cell size balances occupancy; pooled storage aims to reduce garbage collection. These are the author's design reasons, not independently validated optimal choices. The code attempts to retain object picking and inverted-condition behavior, but full equivalence is not established by review or tests in this PR. [PR record](#) · [Changed files](#)

Validation and outcome

The author reports repeated benchmark comparisons, including approximately 43.29 ms to 3.417 ms at 1,000 objects. These are self-reported measurements, not independently reproduced here. The PR was merged by killerdevildog into killerdevildog/GDevelop on 2026-02-17, with no outside review. The final diff includes no benchmark suite, and negative-coordinate correctness remains unestablished: minimum-cell conversion truncates toward zero while maxima use floor. [PR record](#) · [Changed files](#)

SOURCE RECORD

[Pull request description and merge record](#) — killerdevildog; GitHub merge record, 2026-02-17

[Accepted changed-file diff](#) — PR final diff, 2026-02-17

[Contribution commit](#) — Quaylyn Rimer, 2026-02-17

bkaradzic/bgfx · #3593

Coalescing free blocks during deallocation

Pull request author · upstream contribution

Opened 2026-02-14 Merged 2026-02-17

Merged upstream

The problem

NonLocalAllocator tracked free ranges using a list-shaped interface, inserted free blocks without maintaining address order, and deferred sorting and coalescing until compact(). The PR identifies extra element shifts in the default tinystl implementation and repeated merge work. [PR record](#)

Why it was needed

The allocator serves dynamic graphics buffers and the uniform cache. Repeated allocate/free activity can produce multiple neighboring free ranges, increasing entries scanned by first-fit allocation. Keeping adjacent ranges merged addresses that bookkeeping overhead while retaining the existing allocator interface. [PR record](#)

My contribution and the accepted change

Changed FreeList to `std::vector<Free>`. A new `insertFreeBlock` uses `bx::upperBound` to locate the address position and checks whether the block joins the previous, next, both or neither neighbor. `add` and `free` call this helper. `compact` now only reports whether `m_used` is empty, because merging is performed during insertion. `remove` erases the first vector entry; `Free` gains `operator<` for the foundation library comparator. [Changed files](#)

Why this approach

Documented: immediate coalescing keeps the free list compact, address ordering avoids later sort passes, and `bx::upperBound` follows project library conventions without introducing `std` containers. The body attributes the search choice to a maintainer suggestion, but no corresponding review thread is present. Binary search is $O(\log n)$; vector insertion and erasure can still move $O(n)$ elements. First-fit allocation remains linear and now chooses the lowest-address fit. [PR record](#) · [Changed files](#)

Validation and outcome

The author reports a Vulkan headless churn benchmark using 256 buffers and 2,000 measured frames, approximately 0.29 to 0.135 ms average frame time. These workload-specific measurements were not reproduced here, and no benchmark or tests appear in the single-file diff. bkaradzic merged the change on 2026-02-17. [PR record](#) · [Changed files](#)

SOURCE RECORD

Pull request description and merge record — killerdevildog; GitHub merge record, 2026-02-14

Accepted changed-file diff — PR final diff, 2026-02-17

Contribution commit — Quaylyn Rimer, 2026-02-14

pypa/wheel · #669

Inspecting wheel archives from the command line

Pull request author · upstream contribution

Opened 2025-07-19

Merged 2026-02-15

Merged upstream

The problem

Issue #639 requested a way to inspect wheel metadata without unpacking an archive. A user examining a local .whl needed convenient access to package identity, compatibility tags, dependencies, provenance, and contained-file information. [PR record](#)

Why it was needed

The new command makes package inspection available within the existing wheel CLI. The documentation identifies checking contents before installation, understanding supported Python/platform combinations, and debugging packaging as intended uses. The accepted implementation takes a local file path; it does not implement remote PyPI lookup. [PR record](#)

My contribution and the accepted change

Added wheel info with optional -v/--verbose, backed by WheelFile and email BytesParser. It reads WHEEL and METADATA directly, shows filename-derived identity/build data, metadata, tags, dependencies, file count and uncompressed size, and optionally each file. It prints missing-metadata warnings to stderr and translates missing paths through the CLI error mechanism. Documentation, command registration, release notes, and tests accompany the implementation. [Changed files](#)

Why this approach

Documented rationale: direct archive reading avoids extraction. Review led to preserving every Generator field as repeated lines, supporting provenance when multiple tools rewrite a wheel. agronholm agreed with repeated lines and explicitly deferred JSON output to another PR. Tests were revised to avoid depending on argv[0] for help output and to update RECORD hashes when altering fixture metadata. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

Committed tests cover basic and verbose output, generator presence/absence/multiplicity, missing files, help, and the short flag. The author reported passing Python 3.12 and 3.14 tests after addressing review. Codecov reported 88.23529% patch coverage on a named snapshot; it also warned its comparison was behind main, so this is not universal coverage evidence. Merged February 15, 2026. Accepted scope is human-readable local-wheel inspection; JSON remained a requested follow-up. The review history substantiates iterative test, formatting, and output-design improvements. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

Add wheel info subcommand — [pull request and author description](#) — killerdevildog, 2025-07-19

Accepted implementation — [changed files](#) — pypa/wheel, 2026-02-15

Merge commit — agronholm, 2026-02-15

Review discussion by agronholm — agronholm, 2026-02-15

Review discussion by killerdevildog — killerdevildog, 2026-02-15

Review discussion by agronholm — agronholm, 2026-02-15

nem0/LumixEngine · #1538

Batching GPU profiler queue removal

Pull request author · upstream contribution

Opened 2026-02-15 Merged 2026-02-15

Merged upstream

The problem

GPUProfiler::frame() repeatedly removed the first element of `m_queries` while consuming ready queries. Each `erase(0)` moved the remaining array elements, so processing a long ready prefix could repeat substantial copying. [PR record](#)

Why it was needed

Profiling should not introduce avoidable overhead into the rendering frame it observes. With `n` ready queries in an array, repeated front removals can cause quadratic element movement; the operation needs only to discard one processed prefix while retaining queries not ready yet. [PR record](#)

My contribution and the accepted change

Introduced a processed cursor, read `m_queries[processed]`, and incremented it after each successful query handling. After the loop, one guarded `eraseRange(0, processed)` discards the prefix. The existing early stop at the first unready query remains, as do timestamp handling, GPU block reporting, the mutex guard and returning query handles to the pool. [Changed files](#)

Why this approach

Documented: batch the erasure to eliminate repeated memmoves. The source supports the inference that this preserves the queue order and readiness boundary, since only indexing and removal timing change. It reduces array compaction work without introducing a new queue container or changing callers. Linear overall work is a structural claim about processing and compaction, not a universal timing guarantee. [PR record](#) · [Changed files](#)

Validation and outcome

The author reports a median microbenchmark for 1,024 ready queries: 329.61 microseconds before and 0.90 afterward, approximately 366× for that case. The benchmark was not independently rerun. nem0 merged the contribution on 2026-02-15 and acknowledged it in the discussion. Only `src/renderer/renderer.cpp` is in the accepted file list. Merged 2026-02-15 by nem0 into nem0/LumixEngine:master. [PR record](#) · [Changed files](#)

SOURCE RECORD

[Pull request description and merge record](#) — killerdevildog; GitHub merge record, 2026-02-15

[Accepted changed-file diff](#) — PR final diff, 2026-02-15

[Contribution commit](#) — Quaylyn Rimer, 2026-02-15

ArthurSonzogni/FTXUI · #1188

Reducing terminal screen serialization overhead

PR author · performance work with maintainer

Opened 2026-02-15 Merged 2026-02-15

Merged upstream

The problem

FTXUI constructed terminal screen output with stringstream insertions and intermediate color strings. The author targeted the formatting, stream-management and allocation overhead incurred repeatedly while serializing pixels and ANSI styles. [PR record](#)

Why it was needed

Terminal rendering repeatedly converts a screen to escape sequences and text. Reducing per-pixel serialization overhead can improve rendering throughput, especially where style transitions generate many small fragments. The author's benchmark was an 80×24 screen in a ten-second Release run, making its numerical result specific to that workload. [PR record](#)

My contribution and the accepted change

The initial contribution replaced streams with a reserved string in `Screen::ToString`, `ResetPosition` and `UpdatePixelStyle`, and added `Color::PrintTo` to append directly into that output. The final accepted diff also contains maintainer followups: direct small-number formatting, single-byte width shortcuts and ASCII width handling. The changelog expressly credits `killerdevildog` for initiating the preallocation part. [Changed files](#)

Why this approach

Documented: reserving about 30 bytes per screen pixel reduces output growth; direct append avoids stream insertion overhead. During review a suggestion to use `std::format_to` was declined because ArthurSonzogni still supported C++17. The final approach therefore stays compatible with that baseline. The maintainer's additional width work is separate from the original contributor's optimization. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The author reports stable average throughput of 981 to 1041 FPS, about 6.1%. ArthurSonzogni published before/after benchmark tables and then further improvements, noting width-call avoidance as the main driver. The final changelog reports larger combined improvements, roughly 27% for basic/text and 38% styled rendering. The PR merged on 2026-02-15. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — `killerdevildog`; GitHub merge record, 2026-02-15

[Accepted changed-file diff](#) — Changes reviewed at PR head, 2026-02-15

[Original contributor optimization](#) — Changes reviewed at PR head, 2026-02-15

[Maintainer benchmark comparison](#) — ArthurSonzogni, 2026-02-15

[Maintainer additional optimizations](#) — ArthurSonzogni, 2026-02-15

[C++17 compatibility rationale](#) — ArthurSonzogni, 2026-02-15

bkaradzic/bgfx · #3578

Compiling HLSL shaders on Linux with Wine

Integration contributor · existing d3d4linux bridge

Opened **2026-02-04**Merged **2026-02-06**

Merged upstream

The problem

Legacy HLSL compilation in shaderc was enabled for Windows, preventing a Linux build pipeline from producing the same Direct3D shader bytecode through that path. Integrating d3d4linux also exposed incomplete reflection serialization: compiled shaders could lack the uniform type data bgfx needed. [PR record](#)

Why it was needed

The PR sought to compile Direct3D shaders from Linux using Wine and D3DCompiler. Compilation alone was insufficient: constant-buffer names, types and sizes must also cross the process boundary so bgfx can build correct uniform metadata. [PR record](#)

My contribution and the accepted change

Integrated Sam Hocevar's d3d4linux bridge, added shaderc build includes and Linux HLSL configuration, connected compiler/reflection/disassembly/strip callbacks, and added executable-relative bridge/DLL discovery and failure handling. The contribution repaired reflection type serialization/deserialization and disassembly message framing. The final client forcibly marks reflected uniforms used. Maintainer-provided cleanup patches contributed to the final integration. [Changed files](#)

Why this approach

Documented: reuse the Wine bridge and existing shaderc entry points, rather than create a shader compiler; match the data exchanged by both sides of the IPC protocol. bkaradzic identified missing GetType/GetDesc serialization and requested dependency fixes be separated upstream or maintained in a fork. The author subsequently reported adding the missing type descriptor and tested uniform extraction. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The author reports Ubuntu 24.04/Wine 11.0 shader compilation and then a cubes run under Wine/DXVK after verifying u_modelViewProj as a 4×4, 64-byte matrix. bkaradzic merged on 2026-02-06. These are recorded author tests, not new executions. Final SHADERC_CONFIG_HLSL enables Windows and Linux; macOS scaffolding remains but macOS was removed from the default. Merged 2026-02-06 by bkaradzic into bkaradzic/bgfx:master. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

Pull request description and merge record — killerdevildog; GitHub merge record, 2026-02-04

Accepted changed-file diff — Changes reviewed at PR head, 2026-02-06

Maintainer identifies missing reflection data — bkaradzic, 2026-02-04

Author reports reflection fix and runtime test — killerdevildog, 2026-02-04

Maintainer patch attribution in integration commit — Changes reviewed at PR head, 2026-02-06

Final commit removes macOS default — Changes reviewed at PR head, 2026-02-06

[queer/boxxy](#) · #279

Replacing an obsolete configuration example

Documentation contributor

Opened **2026-01-13** Merged **2026-01-13****Documentation merged upstream**

The problem

The README introduced boxxy using relocation of tmux configuration into an XDG-style directory. The contribution identified that example as outdated because newer tmux already supported that location, weakening the explanation of why a redirection tool was useful. [PR record](#)

Why it was needed

An introductory example should demonstrate a concrete need that the tool addresses. An obsolete demonstration can confuse readers about whether installing and configuring the tool adds value. The accompanying old screencast repeated the same example. [PR record](#)

My contribution and the accepted change

Replaced the tmux file rule with an AWS CLI directory redirection from `~/.aws` to `~/.config/aws`. Removed the tmux asciinema embed and expanded the explanation to include control of application read/write locations, project-specific configuration and contextual redirection. All accepted changes are in README.md; no runtime namespace or filesystem behavior changed. [Changed files](#)

Why this approach

Documented: choose an example whose default storage still illustrates redirection, and explain uses beyond XDG compliance. The new rule redirects a directory, so the old explicit file-mode example is removed. The practical inference is that readers see a more relevant initial setup and a clearer connection between the example and the tool's broader purpose. [PR record](#) · [Changed files](#)

Validation and outcome

queer merged the PR on 2026-01-13 and thanked the contributor. The patch and commit verify the documentation update, but contain no executable tests, command transcript or usability study. Any benefit to onboarding is a reasonable editorial objective, not a measured result. Merged 2026-01-13 by queer into `queer/boxxy:mistress`. [PR record](#) · [Changed files](#)

SOURCE RECORD

Pull request description and merge record — [killerdevildog](#); GitHub merge record, 2026-01-13

Accepted changed-file diff — [PR final diff](#), 2026-01-13

Contribution commit — [Quaylyn Rimer](#), 2026-01-13

g3n/engine · #317

Detecting NaN in ray-plane intersections

Pull request author · upstream contribution

Opened 2026-01-07

Merged 2026-01-09

Merged upstream

The problem

Ray.IntersectPlane compared the returned distance with NaN() using equality. A NaN value does not compare equal to itself, so this condition could not detect the no-intersection sentinel it was intended to handle. [PR record](#)

Why it was needed

The method's control flow returns nil when the plane-distance calculation produces NaN. A broken sentinel check lets invalid distance data continue into the subsequent intersection calculation instead of taking that return path, undermining the routine's handling of non-intersections. [PR record](#)

My contribution and the accepted change

Replaced if t == NaN() with if isNaN(t) in math32/ray.go. This uses the package's existing predicate and retains the original nil return and surrounding algorithm. The accepted diff changes one expression in one file, with no API or new dependency. [Changed files](#)

Why this approach

Documented: the PR and commit explicitly explain IEEE 754 NaN equality behavior and select isNaN as the appropriate existing operation. The source supports a narrow rationale: fix the classification test directly rather than modify the geometry calculation or introduce a different sentinel convention. This small change aligns the implementation with the branch it already intended to take. [PR record](#) · [Changed files](#)

Validation and outcome

danaugrs merged the change on 2026-01-09 and acknowledged it. The accepted source and merge are verified. No regression test or execution record accompanies the PR. Merged 2026-01-09 by danaugrs into g3n/engine:master. [PR record](#) · [Changed files](#)

SOURCE RECORD

[Pull request description and merge record](#) — killerdevildog; GitHub merge record, 2026-01-07

[Accepted changed-file diff](#) — PR final diff, 2026-01-09

[Contribution commit](#) — Quaylyn Rimer, 2026-01-07

[badges/shields](#) · #11227

Designing a contributor-recognition badge proposal

Proposal author · API and product design

Opened **2025-07-20** Closed **2025-11-22****Completed discussion** · external-project direction

The problem

I proposed contributor badges that communicate patterns such as language specialization, bug fixing, feature work, maintenance, and sustained work within an organization. The proposal addressed a perceived gap between general activity graphs and more specific descriptions of contribution. [Issue report](#)

Why it was needed

The stated motivation was positive recognition for contributors and a clearer summary for maintainers and project dashboards. These were proposed benefits, not measured outcomes. The issue described configurable thresholds, visual categories, and a positive-recognition-only approach with an opt-out. [Issue report](#)

My contribution and the resolution

I wrote the feature proposal, illustrated badge examples, sketched contribution-grouping pseudocode, and proposed a contributor-badges endpoint using GitHub metadata and issue labels. Maintainer PyvesB explained that a recognition system should be a separate project using Shields.io underneath rather than a new system built into Shields itself. [Issue report](#) · [Discussion](#)

Why this approach

My proposed implementation separated contribution retrieval, categorization, threshold checks, and badge creation. Caching, rate limiting, and webhook updates were included as operational considerations. The maintainer's alternative keeps the policy for awarding achievements outside a service focused on rendering badges. Both positions are documented; no implementation is claimed. [Issue report](#) · [Discussion](#)

Validation and outcome

Although GitHub classifies the closed issue as completed, the proposal was not adopted as an in-house feature. This page is included to account for every completed issue requested, and records a design discussion and project-boundary decision. There is no merged fix, launched endpoint, or verified engagement improvement to attribute to this issue. [Discussion](#)

SOURCE RECORD

Original report · [badges/shields #11227](#) — Quaylyn Rimer / GitHub, 2025-07-20

Maintainer scope decision — PyvesB / Shields.io, 2025-11-16

[encode/django-rest-framework](#) · #9742

Avoiding shared mutable ordering context

Fix author · lint addition by browniebroke

Opened 2025-07-18 Merged 2025-10-30

Merged upstream

The problem

`OrderingFilter.get_default_valid_fields` and `get_valid_fields` used `context={}` as a default argument. The same dictionary object could be reused across calls made without context, a mutable-default pattern highlighted in issue #9741. [PR record](#)

Why it was needed

The cleanup removes possible shared default state and establishes a lint guard against recurrence. Reviewer browniebroke cautioned that normal framework use appears to pass context explicitly, so the evidence supports defensive maintenance rather than a demonstrated widespread data-leak or production failure. [PR record](#)

My contribution and the accepted change

Changed both method defaults to `None` and constructs an empty dictionary inside each method when needed. The accepted PR also adds `flake8-bugbear` to pre-commit dependencies and configures `B006` specifically, leaving other bugbear checks disabled to avoid unrelated existing violations. The added lint configuration came from a separate maintainer commit. [Changed files](#)

Why this approach

Documented design discussion considered whether explicitly passing `None` should remain distinguishable from omitting context. peterthomassen traced context through `Serializer` and `Field`, noted its documented dictionary contract, and supported the current approach. Alternative sentinels or keyword-presence checks were mentioned but not adopted. This reasoning is stronger than the original body's blanket no-breaking-changes claim. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The author reports existing filter tests and a custom verification script passing. Review requested regression tests, but no new test file appears in the accepted diff. The lint rule supplies ongoing static prevention. Reviewers agreed the `None` conversion was acceptable given the dictionary contract; this is a reviewed compatibility judgment, not proof for every downstream subclass. Merged October 30, 2025, following approvals including browniebroke, peterthomassen, and auvipy. Accepted outcome: per-call default dictionaries plus a narrowly enabled mutable-default lint check. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

[Fix mutable default arguments in OrderingFilter methods](#) — pull request and author description — killerdevildog, 2025-07-18

[Accepted implementation](#) — changed files — [encode/django-rest-framework](#), 2025-10-30

[Merge commit](#) — auvipy, 2025-10-30

[Review discussion by peterthomassen](#) — peterthomassen, 2025-10-30

[Review discussion by killerdevildog](#) — killerdevildog, 2025-08-10

[Review discussion by browniebroke](#) — browniebroke, 2025-10-20

opencv/opencv · #27620

Correcting generated Scalar return annotations

PR author · final refinement by Vadim Levin

Opened 2025-08-01 Merged 2025-09-10

Merged upstream

The problem

OpenCV Python typing used `Scalar`, an alias accepting a sequence or a single float, for return values that are actually four-element tuples. This overly broad return annotation caused static-type-checking problems for callers using functions such as `mean`, `trace`, and `sumElems`. [PR record](#)

Why it was needed

Input convenience and return precision have different requirements. A broad accepted input type is useful for drawing functions, but returning a known four-float tuple should not force callers or type checkers to account for a lone float. Correct stubs improve the match between Python runtime behavior and static analysis. [PR record](#)

My contribution and the accepted change

The final accepted diff adds `convert_returned_scalar_to_tuple` to `api_refinement.py` and calls it during manual refinement. It builds a `TupleTypeNode` containing four float nodes and replaces `Scalar` return nodes for function overloads in the root and its namespaces. The existing `Scalar` alias remains unchanged. This final scope differs substantially from the initial `ScalarInput/ScalarOutput` proposal described in the PR body. [Changed files](#)

Why this approach

Documented review rationale: per-function manual input/output changes were suboptimal; the discussion explored overload generation. The final maintainer commit instead targets returned `Scalar` values based on `pyopencv_from` conversion behavior. The architecture of the accepted patch is a global refinement of return nodes rather than a handwritten list of drawing functions. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The author reported building successfully and testing in a Python virtual environment during iteration. A later commit removes the test file, and the final diff contains no regression-test addition. Maintainer review explicitly documents before/after generated return signatures. `asmorkalov` merged the PR on 2025-09-10. The accepted outcome is precise four-float return annotations; it is not the full initial input-type redesign. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

Pull request description and merge record — [opencv/opencv](#), 2025-08-01

Accepted changed-file diff — [opencv/opencv](#), 2025-09-10

Commit: fix: returned `Scalar` is always a tuple of 4 floats — [opencv/opencv](#), 2025-09-10

Discussion by [VadimLevin](#) — [opencv/opencv](#), 2025-08-11

Discussion by [killerdevildog](#) — [opencv/opencv](#), 2025-08-12

Discussion by [VadimLevin](#) — [opencv/opencv](#), 2025-09-10

methusalah/OpenRTS · #117

Modernizing the OpenRTS build and navigation

Pull request author · upstream contribution

Opened 2025-08-16 Merged 2025-09-05

Merged upstream

The problem

The PR describes an aging Java 8 / Gradle 4.9 build, obsolete MaxPermSize configuration, retired Bintray repositories, legacy JMonkeyEngine physics dependencies, and network controls that were not wired to functional handlers. Event serializer registration happened too late for the networking registry. [PR record](#)

Why it was needed

These issues made the game and editor harder to build on a modern Java environment and left navigation and hosting controls incomplete. The contribution addresses buildability and access to existing application modes, rather than introducing a new RTS engine. [PR record](#)

My contribution and the accepted change

The accepted diff selects Java 21, Gradle 8.10.2, JMonkeyEngine 3.8.1, Minie 8.1.0 and Libbulletjme 21.2.1. It replaces obsolete dependency configuration and repositories, adds Game/Editor/Network tabs using ControllerChangeEvent, registers Event classes during application initialization, implements create/join/exit handlers, and adapts physics-space construction and convex sweep calls. A default Ramp constructor supports Jackson deserialization. The create handler still pauses its caller for two seconds after starting the server thread. [Changed files](#)

Why this approach

Documented rationale: modernize the dependency stack while preserving existing assets, maps, and keyboard navigation. Moving serialization earlier addresses registry locking; reusing controller events connects buttons to the established mode-switching mechanism. The author explicitly lists Java 21 as a new minimum requirement. [PR record](#) · [Changed files](#)

Validation and outcome

The author reports manually exercising the three modes, physics, and server/client connection, with instructions to run Gradle and try the tabs and network workflow. No independent benchmark or automated regression-test results are attached in the reviewed PR evidence. Pesegato merged the contribution upstream on 2025-09-05. The merge verifies acceptance, not every broad compatibility or performance statement in the description. [PR record](#) · [Changed files](#)

SOURCE RECORD

Pull request description and merge record — methusalah/OpenRTS, 2025-08-16

Accepted changed-file diff — methusalah/OpenRTS, 2025-09-05

Commit: Modernize to Java 21 and JME 3.8.1 with Minie physics — methusalah/OpenRTS, 2025-08-16

Amphuse76/GoGetEm · #1

Updating a Godot scene to a green sphere

Pull request author · upstream contribution

Opened 2025-08-18 Merged 2025-08-18

Merged upstream

The problem

The only stated change request is the PR title, changed to green sphere. The public PR has no body or problem report explaining a defect, user requirement or gameplay goal. Before the change, the scene's sphere material override was magenta. [PR record](#)

Why it was needed

The available evidence supports a visual scene revision for the GoGetEm project. A broader need such as player visibility, accessibility, collision tuning or game balance is not documented and cannot responsibly be supplied as the author's motivation. [PR record](#)

My contribution and the accepted change

Modified the Godot node_3d.tscn scene: changed StandardMaterial3D albedo from Color(1, 0, 1, 1) to a green-toned value, added a second StandardMaterial3D subresource, assigned it as the CSGSphere3D material, and set the sphere radius to 16.69. The scene resource load-step count increased from two to three to account for the added material. The original material override remains assigned with its changed color. [Changed files](#)

Why this approach

Documented rationale is limited to the intended green appearance expressed by the title and commit message. An implementation-level inference is that editing the existing declarative scene resource directly stores the desired appearance and geometry without adding runtime scripting. There is no evidence that the extra material or particular radius was chosen for a specific engineering tradeoff. [PR record](#) · [Changed files](#) · [Discussion](#)

Validation and outcome

Amphuse76 approved the PR on 2025-08-18 and merged it shortly afterward into Amphuse76/GoGetEm. The final diff and approval establish acceptance by an independently owned project. No screenshot, scene execution log, discussion or automated test appears in the examined PR. Merged 2025-08-18 by Amphuse76 into Amphuse76/GoGetEm:main. [PR record](#) · [Changed files](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — killerdevildog; GitHub merge record, 2025-08-18

[Accepted changed-file diff](#) — Changes reviewed at PR head, 2025-08-18

[Contributor intent recorded in commit title](#) — Changes reviewed at PR head, 2025-08-18

[Project owner approval](#) — Amphuse76, 2025-08-18

OpenKore/openkore · #4070

Making maximum walking distance configurable

Pull request author · upstream contribution

Opened 2025-08-16 Merged 2025-08-16

Merged upstream

The problem

Field.canMove rejected destinations whose block distance exceeded a hardcoded 17. An existing source comment said this corresponded to the server max_walk_path setting, which may be configured above its default, and left a TODO to make the client value configurable. [PR record](#)

Why it was needed

A fixed client-side limit could prevent movement that a configured server permits. Exposing the setting lets deployments align the client path-distance check with their server without editing Perl source, while retaining familiar behavior for existing installations. [PR record](#)

My contribution and the accepted change

The patch imports %config from Globals into Field.pm, assigns \$config{maxWalkPathDistance} || 17 to a local maximum, and compares the computed blockDistance against it. control/config.txt documents maxWalkPathDistance and supplies 17 as the shipped default. The rest of the canMove algorithm is unchanged by this contribution. [Changed files](#)

Why this approach

Documented rationale: resolve the TODO and mirror the server setting while preserving the default limit. Code-level interpretation: keeping the same fallback limits migration work for configurations that do not contain the new key. The use of Perl logical-or also treats a zero value as absent; the patch does not add a new numeric validation layer. [PR record](#) · [Changed files](#)

Validation and outcome

The reviewed snapshot contains the single implementation commit and an upstream review by alisonrag, but no added test files, reported command output, or benchmark. The diff and merge establish the configuration change; comprehensive routing tests are not documented. alisonrag merged PR #4070 upstream on 2025-08-16, the day it was opened. Accepted scope is configuration of the existing distance cutoff. [PR record](#) · [Changed files](#)

SOURCE RECORD

Pull request description and merge record — OpenKore/openkore, 2025-08-16

Accepted changed-file diff — OpenKore/openkore, 2025-08-16

Commit: Make maxWalkPath configurable in Field.pm — OpenKore/openkore, 2025-08-16

tornadoweb/tornado · #3523

Updating the file-upload demo to native async

Pull request author · upstream contribution

Opened 2025-07-28

Merged 2025-08-05

Merged upstream

The problem

The `file_uploader.py` example could raise `ValueError` when its `gen.coroutine`-decorated entry points were passed to `asyncio.run`. The PR explains that these decorated functions produced Tornado Future objects whereas the relevant `asyncio.run` interface expected native coroutine objects. [PR record](#)

Why it was needed

A broken sample prevents users from trying the documented upload workflow and teaches an incompatible combination of old and modern asynchronous styles. Repairing the example restores a practical starting point for multipart POST and raw PUT uploads. [PR record](#)

My contribution and the accepted change

The accepted diff removes the `tornado.gen` import and converts `multipart_producer`, `post`, `raw_producer`, and `put` to `async def` functions. It replaces `yield` expressions around streamed writes and `AsyncHttpClient.fetch` with `await`. The multipart boundaries, headers, URL construction, per-file processing, and 16 KiB chunked reads remain structurally the same. [Changed files](#)

Why this approach

Documented rationale: use native `async/await` so the entry points match `asyncio.run` while retaining asynchronous body production. Code-level interpretation: converting producers together with callers keeps awaiting consistent across the upload chain instead of merely wrapping the top-level returned Future. The change is confined to the example, not Tornado's coroutine API or HTTP client implementation. [PR record](#)

[Changed files](#) · [Discussion](#)

Validation and outcome

The reviewed PR contains one implementation commit and a short maintainer acknowledgment, with no added test files or execution transcript. The diff demonstrates the intended coroutine conversion. The title and description identify #3182 as the reported issue. bdarnell merged PR #3523 on 2025-08-05 and thanked the contributor. The accepted scope is modernization and compatibility of this upload demo. [PR record](#) · [Changed files](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — tornadoweb/tornado, 2025-07-28[Accepted changed-file diff](#) — tornadoweb/tornado, 2025-08-05[Commit: Fix ValueError in file_uploader.py by converting @gen.coroutine to async/await](#) — tornadoweb/tornado, 2025-07-28[Discussion by bdarnell](#) — tornadoweb/tornado, 2025-08-05

matrixcascade/PainterEngine · #144

Improving WebAssembly keyboard character mapping

Pull request author · upstream contribution

Opened 2025-08-01 Merged 2025-08-04

Merged upstream

The problem

The PR describes WebAssembly keyboard input producing a plain 2 for Shift+2, incorrect Shift+9 output, and unexpected characters for numeric-keypad digits. The previous keydown path assigned the SDL key symbol directly to a one-character buffer before handling special keys. [PR record](#)

Why it was needed

Users need recognizable characters in browser-hosted engine interfaces. Modifier-aware handling can translate shifted US number-row keys and common punctuation into their intended printable characters instead of exposing raw key-code behavior. [PR record](#)

My contribution and the accepted change

The accepted main.c patch reads `SDL_GetModState` and tests `KMOD_SHIFT`. After existing special-key cases it adds a numeric-keypad branch, a shifted number-row lookup table, mappings for common shifted punctuation, uppercase conversion for Latin letters, and a printable ASCII fallback. The existing downstream event dispatch remains in place. [Changed files](#)

Why this approach

Documented rationale: address the reported shift-number and keypad problems while retaining the existing event model. Code-level interpretation: explicit tables make the intended US-layout punctuation mappings easy to inspect. The implementation is a keycode-to-character mapping, not a general composition or international text-input system. [PR record](#) · [Changed files](#)

Validation and outcome

The PR and commit claim comprehensive validation and passing cases, but the reviewed change contains no accompanying test files or reproducible test transcript. The shift-number table visibly maps 2 to @ and 9 to an opening parenthesis. The keypad branch uses a range from `SDLK_KP_0` to `SDLK_KP_9` and arithmetic on key symbols; complete keypad coverage was not independently verified. matrixcascade merged the contribution upstream on 2025-08-04. Acceptance is verified from the PR record. Broader keyboard-layout correctness is not established by that event. [PR record](#) · [Changed files](#)

SOURCE RECORD

[Pull request description and merge record](#) — matrixcascade/PainterEngine, 2025-08-01

[Accepted changed-file diff](#) — matrixcascade/PainterEngine, 2025-08-04

[Commit: Fix web keyboard input for shift+number and numpad keys](#) — matrixcascade/PainterEngine, 2025-08-01

[playcanvas/engine](#) · #7877

Removing a broken contributor-agreement step

Issue reporter · contributor onboarding

Opened **2025-08-01** Closed **2025-08-04****Resolved through maintainer PR**

The problem

The contribution guide directed contributors to a Google Forms contributor-agreement link that I reported was no longer working. The same agreement appeared in the PR template, making the broken destination part of the expected submission process. [Issue report](#) · [Resolution](#)

Why it was needed

A required but inaccessible onboarding step creates uncertainty for contributors about whether a submission is ready for review. My report identified the exact failing destination. The inspected follow-up shows that the link was embedded in both the guide and the template, rather than being an isolated page reference. [Issue report](#) · [Discussion](#)

My contribution and the resolution

I filed the broken-link report. Maintainer willeastcott authored PR #7883, explicitly linked it to my issue, and removed the CLA step from both documents while expanding setup, test, and submission guidance. This case credits me with discovering and reporting the obstacle; the documentation rewrite belongs to the maintainer. [Issue report](#) · [Resolution](#) · [Discussion](#)

Why this approach

The accepted change removes the obsolete requirement instead of substituting another form URL. Applying the change to both entry points makes the workflow consistent. That consistency is an inference from the patch; the maintainer's PR explicitly describes removal of the CLA but does not give a longer policy rationale. [Resolution](#) · [Discussion](#)

Validation and outcome

PR #7883 merged on August 4, 2025 and closed #7877. The final diff removes the form link from both affected files. No runtime behavior or product performance improvement is claimed: this is a verified contribution to the clarity of the upstream contribution process. [Resolution](#) · [Discussion](#)

SOURCE RECORD

Original report · [playcanvas/engine #7877](#) — Quaylyn Rimer / GitHub, 2025-08-01

Linked maintainer PR #7883 — willeastcott / PlayCanvas, 2025-08-04

Accepted guide and template changes — PlayCanvas, 2025-08-04

phase1geo/Minder · #659

Guarding image resizing against invalid dimensions

Pull request author · upstream contribution

Opened 2025-08-03 Merged 2025-08-03

Merged upstream

The problem

The linked crash report, as described in the PR, showed image-resize failures with GdkPixbuf assertions about nonpositive destination height and invalid pixbuf access, followed by GTK warnings. `NodeImage.set_width` previously calculated scaling and assigned its result without guarding crop dimensions or extraction/scaling failure. [PR record](#)

Why it was needed

An image-editing gesture should not cascade into invalid graphics-buffer operations. Defensive checks at the resize boundary can preserve the current displayed image when the requested dimensions or intermediate buffer cannot be used. [PR record](#)

My contribution and the accepted change

The accepted Vala patch returns early for nonpositive crop width or height and for nonpositive target width. It verifies that `pixbuf_get_from_surface` returns a buffer, clamps a calculated nonpositive scaled height to one pixel, and assigns `_buf` only if `scale_simple` succeeds. Warning messages identify rejected dimensions and extraction failure; a failed scale retains the previous buffer. [Changed files](#)

Why this approach

Documented rationale: prevent division by zero and invalid GdkPixbuf arguments and stop failure propagation from image resizing. The intervention is intentionally local to `NodeImage.set_width` and keeps the existing bilinear interpolation strategy. The retained buffer supplies a graceful fallback rather than forcing a replacement image. [PR record](#) · [Changed files](#) · [Discussion](#)

Validation and outcome

The author explicitly could not reproduce the exact crash sequence from the report video on current master. Their analysis used the visible terminal diagnostics. No automated test is included. The patch adds defensive checks for the observed failure modes; an end-to-end reproduction of the original crash is not documented. phase1geo merged the change on 2025-08-03 and stated that it was committed to master and would be forward-ported to the development branch. [PR record](#) · [Changed files](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — phase1geo/Minder, 2025-08-03

[Accepted changed-file diff](#) — phase1geo/Minder, 2025-08-03

[Commit: Fix image resize crash in set_width method](#) — phase1geo/Minder, 2025-08-03

[Discussion by phase1geo](#) — phase1geo/Minder, 2025-08-03

MyGUI/mygui · #289

Registering existing test applications with CTest

Pull request author · upstream contribution

Opened 2025-08-02 Merged 2025-08-02

Merged upstream

The problem

MyGUI could build its UnitTests applications without enabling CMake testing or registering them for CTest. The PR addresses issue #273 by adding this missing infrastructure. Review also clarified that these applications are interactive demonstrations with event loops, rather than self-terminating assertion suites. [PR record](#)

Why it was needed

Registering executables makes the existing test applications discoverable through CMake tooling and creates a foundation for later automation. It does not itself turn interactive programs into unattended tests, and that distinction was central to the maintainer discussion. [PR record](#)

My contribution and the accepted change

The top-level CMakeLists calls `enable_testing` when `MYGUI_BUILD_UNITTESTS` is enabled. The `mygui_unit_test` helper registers each executable with `add_test` and sets its working directory to `CMAKE_RUNTIME_OUTPUT_DIRECTORY` so resources can be found. Both changes are gated on the existing unit-test option and retain the established executable-building flow. [Changed files](#)

Why this approach

Documented rationale from the author response: land the CMake registration as a focused foundational step, then handle application termination separately. The author suggested possible follow-ups such as a noninteractive mode, timeouts, or real assertion-based tests. The maintainer agreed that starting with registration made sense. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The author explicitly reported trying CTest and finding that the applications hang because they do not terminate automatically. Altren raised this behavior before accepting the scope. No claim that the test suite passes unattended is supported; adding registration also does not prove a make check target exists for every generator. Altren merged PR #289 on 2025-08-02 after discussing the limitation. The result is accepted testing infrastructure, with application automation remaining future work. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — MyGUI/mygui, 2025-08-02[Accepted changed-file diff](#) — MyGUI/mygui, 2025-08-02[Commit: Enable CMake testing infrastructure for unit tests](#) — MyGUI/mygui, 2025-08-02[Discussion by Altren](#) — MyGUI/mygui, 2025-08-02[Discussion by killerdevildog](#) — MyGUI/mygui, 2025-08-02[Discussion by Altren](#) — MyGUI/mygui, 2025-08-02

raysan5/raylib · #5075

Adding native ARM64 Linux release jobs

Workflow contributor · follow-up repair by raysan5

Opened **2025-07-31** Merged **2025-08-01**

Merged upstream

The problem

The Linux workflow's matrix covered 32-bit x86 and 64-bit x86 but not ARM64. The PR references a request for ARM64 precompiled binaries, so users of that architecture lacked equivalent output from this release workflow.

[PR record](#)

Why it was needed

Native architecture coverage helps distribute ready-built library artifacts alongside existing x86 packages. The contribution extends the project's packaging path rather than claiming a change to raylib rendering performance or a new ARM implementation in the library itself. [PR record](#)

My contribution and the accepted change

The accepted linux.yml diff moves runner selection into matrix entries and adds an aarch64 entry using ubuntu-24.04-arm with the arm64 artifact-name component. Existing x86 entries retain ubuntu-latest. It separates 64-bit x86 and ARM steps by matrix architecture and builds both static and shared libraries for the new ARM job while retaining the existing 32-bit step. [Changed files](#)

Why this approach

Documented rationale: use a native ARM runner and preserve the established x86/x64 builds. Code-level interpretation: matrix-specific runners allow the same release workflow to package several CPU architectures while using the platform compiler in each environment. The PR's expected artifact name is a stated intention, not evidence that a particular release file was published. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The maintainer first thanked the author, then explicitly reported that the workflow was not working and linked a failed Actions job. After the author offered to update it, raysan5 stated that he had fixed it. This follow-up matters: the original merge alone did not demonstrate a working final build. raysan5 merged PR #5075 on 2025-08-01. The resulting upstream effort included a maintainer correction after the submitted workflow change. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — raysan5/raylib, 2025-07-31[Accepted changed-file diff](#) — raysan5/raylib, 2025-08-01[Commit: \[build\] Add ARM64 support to Linux builds](#) — raysan5/raylib, 2025-07-31[Discussion by raysan5](#) — raysan5/raylib, 2025-08-01[Discussion by killerdevildog](#) — raysan5/raylib, 2025-08-01[Discussion by raysan5](#) — raysan5/raylib, 2025-08-01

alekmaul/pvsneslib · #312

Updating the Hello World tutorial APIs

Documentation contributor

Opened 2025-08-01 Merged 2025-08-01

Documentation merged upstream

The problem

The Hello World tutorial still told readers to call `consoleInit` explicitly and used outdated `consoleSetTextVramBGAdr` and `consoleSetTextVramAdr` names. The author identified a mismatch between the documentation and the working `hello_world.c` example. [PR record](#)

Why it was needed

An introductory tutorial is often a contributor's first build attempt. Obsolete initialization instructions and renamed functions create confusion before a new SNES developer reaches their own program logic. Aligning the sample and explanation reduces this avoidable onboarding problem. [PR record](#)

My contribution and the accepted change

The accepted `wiki/Compiling-helloworld-example.md` diff removes the explicit `consoleInit` call from the displayed main function. It replaces the old VRAM API names with `consoleSetTextMapPtr` and `consoleSetTextGfxPtr` in every affected snippet. The prose now explains that runtime startup handles SNES initialization and shows the text-console initialization sequence when text output is needed. [Changed files](#)

Why this approach

Documented rationale: make the tutorial match the existing working example and current runtime behavior. The patch preserves the demonstrated text map and graphics addresses, text offset, font pointers, and `consoleInitText` setup. It corrects instructions rather than changing the runtime or introducing a new initialization mechanism. [PR record](#) · [Changed files](#)

Validation and outcome

The reviewed snapshot has two commits: one removes the obsolete initialization requirement and rewrites its explanation; the next updates the renamed functions. No compiler output, fresh hardware test, or automated documentation validation is attached. The author states that the final documentation matches the working example; the inspected diff confirms the specific documentation edits. alekmaul merged the PR on 2025-08-01, shortly after submission. This is an accepted documentation contribution addressing the issue referenced as #309 in the title and first commit. [PR record](#) · [Changed files](#)

SOURCE RECORD

Pull request description and merge record — alekmaul/pvsneslib, 2025-08-01

Accepted changed-file diff — alekmaul/pvsneslib, 2025-08-01

Commit: Update function names to match actual working code — alekmaul/pvsneslib, 2025-08-01

VirtualBox/virtualbox · #83

Correcting repeated spelling errors across VirtualBox

Issue reporter · documentation quality

Opened **2025-07-17** Closed **2025-07-31****Confirmed fixed in upstream trunk**

The problem

I reported repeated instances of “preceeding” and “preceeded” across comments, documentation, and source text, including concrete locations in scmrw.cpp. The report explicitly described a readability defect rather than a functional failure. [Issue report](#)

Why it was needed

Repeated misspellings make technical documentation less consistent and can be copied into additional files. Supplying exact search terms and example locations made the report easy to verify without a specialized runtime environment. No claim of execution-speed, security, or algorithmic impact is supported. [Issue report](#)

My contribution and the resolution

I identified and reported the typo family and described how to reproduce the search. Andreas Loeffler’s upstream commit references gh-83 and changes spelling across 30 files, including headers, debugger and runtime comments, and some diagnostic text. I did not author that implementation commit. [Issue report](#) · [Resolution](#)

Why this approach

A narrow textual correction addresses the reported defect while preserving the surrounding logic. The accepted patch makes the same substitutions across related locations rather than modifying code structure. This explanation is inferred from the diff; the commit message confirms its comment/documentation scope. [Resolution](#)

Validation and outcome

The maintainer replied that the issue should be fixed in trunk and thanked me for the report. The inspected commit carries that issue reference and the corresponding substitutions. The issue closed on July 31, 2025. There is no linked regression-test result for this textual cleanup. [Discussion](#) · [Resolution](#)

SOURCE RECORD

Original report · VirtualBox/virtualbox #83 — Quaylyn Rimer / GitHub, 2025-07-17

Upstream typo corrections — Andreas Loeffler / VirtualBox, 2025-07-31

Maintainer trunk confirmation — alflr / GitHub, 2025-07-31

microsoft/terminal · #19185

Refreshing scrollbar marks from shell integration

Pull request author · upstream contribution

Opened 2025-07-29 Merged 2025-07-29

Merged upstream

The problem

Shell integration sequences created command or prompt marks in the terminal buffer, but those marks did not appear in the scrollbar until the user scrolled manually. The PR identifies missing UI notification, rather than missing mark creation, as the problem. [PR record](#)

Why it was needed

Scrollbar marks are navigation cues for prompts and command boundaries. Delayed visibility made shell-created marks behave differently from automatically detected marks and forced a user action before the display reflected existing buffer state. [PR record](#)

My contribution and the accepted change

The patch adds `NotifyShellIntegrationMark` to `ITerminalApi` and implements it in `Terminal` by calling `_NotifyScrollEvent`. `AdaptDispatch` invokes the notification after `ConEmu OSC 9;12`, `iTerm2 OSC 1337 SetMark`, and `FinalTerm OSC 133 A/B/C/D` actions. Empty conhost support and a test-mock implementation satisfy all implementations of the extended interface. [Changed files](#)

Why this approach

Documented rationale: reuse the existing scroll-notification mechanism immediately after mark updates. The commit series also records fixing abstract-class compilation errors after extending the interface and broadening the original `FinalTerm` coverage to other mark protocols. Reviewers accepted the local repair while noting that synchronous output-thread/buffer/control coupling remained a larger architectural problem. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The PR checklist says tests passed, and Azure Pipelines was explicitly triggered in discussion. The inspected test diff only adds the required mock method, without a new behavioral assertion. The GitHub record confirms reviewer approval and merge, but does not independently substantiate the entire checklist or a documentation PR. DHowett merged PR #19185 on 2025-07-29 and thanked the contributor. The accepted change refreshes the scrollbar through the established notification path. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

[Pull request description and merge record](#) — microsoft/terminal, 2025-07-29

[Accepted changed-file diff](#) — microsoft/terminal, 2025-07-29

[Commit: Add NotifyShellIntegrationMark\(\) to iTerm2 and ConEmu mark actions](#) — microsoft/terminal, 2025-07-29

[Discussion by DHowett](#) — microsoft/terminal, 2025-07-29

[Discussion by lhecker](#) — microsoft/terminal, 2025-07-29

[Discussion by DHowett](#) — microsoft/terminal, 2025-07-29

profanity-im/profanity · #2050

Adapting recipient lookup for newer GPGME

Pull request author · upstream contribution

Opened 2025-07-23

Merged 2025-07-23

Merged upstream

The problem

Profanity failed to build against GPGME 2.0.0 or later because the recipient-address path in `p_gpg_decrypt` still referenced `gpgme_key_get_string_attr` and `GPGME_ATTR_EMAIL`. The PR identifies issue #2048 as the concrete build failure being addressed. [PR record](#)

Why it was needed

The encryption integration needed to remain buildable as its GPGME dependency changed. Extracting an email address was still required when composing the recipient string, so simply removing the unsupported call would discard useful recipient information. [PR record](#)

My contribution and the accepted change

Added the static `_gpgme_key_get_email` helper in `src/pgp/gpg.c` and routed the existing call through it. The helper guards a null key. Its conditional branch retains the legacy expression when `GPGME_ATTR_EMAIL` is available to the preprocessor; the other branch walks `key->uids` and returns the first nonempty email. It returns NULL if none is available and documents that the returned address belongs to GPGME memory. [Changed files](#)

Why this approach

Documented rationale: isolate compatibility handling behind one helper while retaining the calling code's existing optional-address behavior. The fallback inspects user IDs directly rather than depending on the removed accessor. The patch uses `#ifdef` on a symbol, not an explicit numeric GPGME version check; branch selection therefore depends on preprocessor symbol availability. [PR record](#) · [Changed files](#) · [Discussion](#)

Validation and outcome

The author reports testing GPGME 1.18.0, compilation compatibility, resolution of the cited errors, and no functional regression. No separate test file or independently reproduced build log appears in the changed files. jubalh approved and merged the PR on July 23, 2025, and thanked the contributor. The verifiable result is accepted compatibility code; broader runtime or security improvements were not measured. [PR record](#) · [Changed files](#) · [Discussion](#)

SOURCE RECORD

Fix GPGME >= 2.0.0 compatibility issue #2048 — pull request and author description — killerdevildog, 2025-07-23

Accepted implementation — changed files — profanity-im/profanity, 2025-07-23

Merge commit — jubalh, 2025-07-23

Review discussion by jubalh — jubalh, 2025-07-23

nasa/ExoMiner · #2

Validating TIC input CSV structure

Pull request author · upstream contribution

Opened 2025-07-21

Merged 2025-07-23

Merged upstream

The problem

run_pipeline.py contained an unresolved TODO to check the structure of incoming TIC IDs CSV data. Missing columns, empty input, unparseable identifiers, or malformed sector-run strings could enter the pipeline without this explicit structural check. [PR record](#)

Why it was needed

The ExoMiner pipeline expects tic_id and sector_run fields. Reporting malformed input near ingestion gives users a concrete place to correct their CSV rather than leaving errors to surface during later processing. The accepted change concerns input diagnostics and structural validation, not an improvement to the scientific classifier or demonstrated discovery accuracy. [PR record](#)

My contribution and the accepted change

Added validate_tic_ids_csv_structure to exominer_pipeline/utils.py and called it after input preparation in run_pipeline.py. The helper raises SystemExit for absent required columns or an empty dataframe. It attempts integer conversion after removing .0 text from TIC IDs, checks sector_run against a digits-hyphen-digits pattern, and logs row-numbered diagnostics, limiting displayed examples to five per category.

VALIDATION_IMPLEMENTATION.md describes the behavior. [Changed files](#)

Why this approach

Documented intent: replace the existing TODO with an explicit, reusable check and actionable messages. Critical structure errors are separated from row-format warnings. Source-based limitation: row problems return False, but the new caller does not inspect that return value. Consequently the implementation does not halt for every invalid row, despite broader wording in the added documentation. [PR record](#) · [Changed files](#)

Validation and outcome

The PR and implementation document claim valid, malformed, missing-column, and empty-input cases were tested. No executable test file was included among the three changed files. The regex validates shape only; it does not establish ordered or scientifically valid sector bounds. Merged July 23, 2025. Confirmed outcome: structural failures stop execution and row diagnostics are added. Malformed rows can still reach downstream processing because the caller ignores the returned False. [PR record](#) · [Changed files](#)

SOURCE RECORD

Fix tic ids csv structure check — pull request and author description — killerdevildog, 2025-07-21

Accepted implementation — changed files — nasa/ExoMiner, 2025-07-23

Merge commit — migmartinho, 2025-07-23

phase1geo/Minder · #656

Guarding the tab context menu against nulls

Pull request author · upstream contribution

Opened 2025-07-19 Merged 2025-07-20

Merged upstream

The problem

The PR addresses issue #655, a tab right-click crash in Minder's DynamicNotebook. The context-menu path used menu items and menu references without checking that those optional objects existed. [PR record](#)

Why it was needed

Right-clicking a tab is ordinary interface interaction. A defensive fix in the current notebook implementation could make the existing Minder 1.x branch more robust while the maintainer prepared a different notebook architecture for version 2.0. [PR record](#)

My contribution and the accepted change

Added null checks around the context menu and relevant items in `src/DynamicNotebook.vala`. Popup and item updates run only when menu exists; Close Other Tabs, New Window, and Pin/Unpin labels or sensitivity are guarded by their respective objects. Existing tab-count and pinned-state behavior is retained inside those guards. A later hunk normalizes indentation in destroy-signal disconnection code. [Changed files](#)

Why this approach

Implementation-derived rationale: the existing event handler can skip unavailable menu components instead of dereferencing them. It is a localized defensive change rather than a widget redesign. Maintainer phase1geo explicitly said version 2.0 would switch to Gtk4 and deprecate DynamicNotebook, and accepted this fix in case a 1.x bug-fix release was made. [PR record](#) · [Changed files](#) · [Discussion](#)

Validation and outcome

The GitHub record provides no new automated test or documented execution matrix. The maintainer described it as a suitable fix and approved its inclusion for the existing version. That establishes review acceptance, but not an independently verified release containing the patch or a reproduced crash resolution on every desktop environment. Merged July 20, 2025. The result is an accepted guard against missing context-menu objects in the legacy notebook. The maintainer's statement about future Gtk4 is historical planning at review time, not evidence of current release status. [PR record](#) · [Changed files](#) · [Discussion](#)

SOURCE RECORD

Fix tab right-click crash (issue #655): add defensive null checks to ... — pull request and author description — killerdevildog, 2025-07-19

Accepted implementation — changed files — phase1geo/Minder, 2025-07-20

Merge commit — phase1geo, 2025-07-20

Review discussion by phase1geo — phase1geo, 2025-07-20

ultimatepp/ultimatepp · #293

Preserving state before refresh initialization

Pull request author · upstream contribution

Opened 2025-07-19 Merged 2025-07-19

Merged upstream

The problem

RichTxt::SetRefreshFrom assigned `r_type = FROM` before checking whether `r_type == NONE`. That made the `NONE` branch unreachable and prevented the method from using the prior refresh state to decide how to initialize `r_parti`. The PR links issue #137. [PR record](#)

Why it was needed

The method needs to remember the first paragraph requiring refresh. When no refresh is pending, the incoming paragraph should initialize the position; when refresh work already exists, the earlier paragraph should win. Evaluating the state after overwriting it defeats that distinction. [PR record](#)

My contribution and the accepted change

Moved `r_type = FROM` after the conditional in `uppsrc/RichText/TxtData.cpp`. The `NONE` branch now assigns `r_parti = parti`; otherwise `r_parti` becomes `min(parti, r_parti)`. The method then records `FROM`. The changed file also reflects indentation differences, but the functional adjustment is assignment ordering, not a new refresh algorithm. [Changed files](#)

Why this approach

The author explicitly explained in the PR discussion that checking the condition before setting `FROM` makes the logic work as intended. Source-based reasoning: preserving the previous state until after the branch allows first-use initialization while retaining the minimum-position rule for accumulated work. This is a direct state-ordering correction with minimal scope. [PR record](#) · [Changed files](#) · [Discussion](#)

Validation and outcome

No new test file or explicit test results are present in the PR record. The old unreachable branch and reordered statement can be verified directly in the diff. No measured rendering improvements or visual regression results are documented. Merged July 19, 2025. The accepted change repairs the branch ordering in `SetRefreshFrom`. The review record in the record consists of the author explanation and merge, with no detailed maintainer discussion of further design alternatives. [PR record](#) · [Changed files](#) · [Discussion](#)

SOURCE RECORD

Fix RichTxt::SetRefreshFrom logic (issue #137): move `r_type` assignmen... — pull request and author description — killerdevildog, 2025-07-19

Accepted implementation — changed files — ultimatepp/ultimatepp, 2025-07-19

Merge commit — mirek-fidler, 2025-07-19

Review discussion by killerdevildog — killerdevildog, 2025-07-19

psf/requests · #6994

Documenting timeout units in seconds

Documentation contributor

Opened 2025-07-18 Merged 2025-07-18

Documentation merged upstream

The problem

The `Session.request` timeout docstring said how long to wait without naming the units, while the higher-level `requests.request` documentation already identified seconds. The PR links documentation issue #6813. [PR record](#)

Why it was needed

Callers consulting the Session API should be told the unit expected for a numeric timeout. Explicit units reduce ambiguity when choosing either a single float or a connect/read tuple and align related documentation without changing how Requests performs network operations. [PR record](#)

My contribution and the accepted change

Changed one line in `src/requests/sessions.py` from How long to wait to How many seconds to wait. The surrounding explanation of waiting for server data and the float-or-tuple parameter form remained intact. The accepted contribution is a documentation correction, with no change to timeout parsing, connection behavior, retry policy, exception handling, or public function signature. [Changed files](#)

Why this approach

Documented rationale: make `Session.request` match `requests.request` and fix the unit inconsistency. The minimal change puts the missing information directly in the parameter description where readers need it. It does not attempt a larger rewrite of timeout semantics or claim that the value imposes a total wall-clock deadline. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

No new tests or runtime results accompany this one-line docstring edit. nateprewitt approved and thanked the contributor. After merge, DetachHead noted that only one location had been updated and pointed to the existing broader PR #6814. This limits any claim that this PR alone standardized all timeout documentation. Merged July 18, 2025. The verified outcome is explicit seconds in `Session.request`'s parameter documentation. Broader timeout documentation remained separate work. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

[docs: clarify timeout parameter uses seconds in Session.request](#) — pull request and author description — killerdevildog, 2025-07-18

[Accepted implementation](#) — [changed files](#) — psf/requests, 2025-07-18

[Merge commit](#) — nateprewitt, 2025-07-18

[Review discussion by nateprewitt](#) — nateprewitt, 2025-07-18

[Review discussion by DetachHead](#) — DetachHead, 2025-07-18

ezEngine/ezEngine · #1613

Packaging ShaderExplorer assets for Android

Pull request author · upstream contribution

Opened 2025-07-17 Merged 2025-07-18

Merged upstream

The problem

ShaderExplorer's CMake file contained a TODO for actual packaging and embedded Android-specific copy commands for sample data, base shaders, and a Vulkan shader cache. The sample lacked a clean use of the engine's packaging abstraction. [PR record](#)

Why it was needed

Deployed samples need their shader and sample-data assets alongside executable packaging. Sharing platform-specific packaging through the engine's existing API reduces duplication and gives the sample a simpler declaration of its required assets. [PR record](#)

My contribution and the accepted change

Replaced the sample's inline copy commands with `ez_package_files` calls for sample data and base shaders. Vulkan cache packaging is conditional on the directory existing. Added `ez_platformhook_package_files` in `Configure_Android.cmake`: it creates the destination under `package/Assets` and copies directories in `PRE_BUILD`, before existing `POST_BUILD` APK generation. The Android-only `InspectorPlugin` link remains.

[Changed files](#)

Why this approach

Documented review rationale: jankrassnigg asked to implement the Android platform hook, citing the existing Web hook as a model, so the sample file would stay short and clean. A follow-up commit explicitly implements that request. The final architecture centralizes Android details behind the established packaging entry point rather than retaining the body's originally described local fallback. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The author reports a Linux build completing 740 targets, simulated Android packaging checks, 139 copied files, directory checks, and conditional-cache tests. These are author-reported historical results, not tests rerun during research. Simulation is not evidence of a successfully installed Android APK. No separate test artifact is in the two-file final patch. jankrassnigg approved the revised solution, which merged July 18, 2025. The verified outcome is accepted packaging integration and a reusable Android hook. Physical deployment across all platforms was not verified. [PR record](#) · [Changed files](#) · [Discussion](#) · [Discussion](#) · [Discussion](#)

SOURCE RECORD

[Implement proper packaging for ShaderExplorer sample](#) — pull request and author description — killerdevildog, 2025-07-17

[Accepted implementation](#) — [changed files](#) — ezEngine/ezEngine, 2025-07-18

[Merge commit](#) — jankrassnigg, 2025-07-18

[Review discussion by killerdevildog](#) — killerdevildog, 2025-07-17

[Review discussion by jankrassnigg](#) — jankrassnigg, 2025-07-17

[Review discussion by jankrassnigg](#) — jankrassnigg, 2025-07-18

cycfi/elements · #438

Removing duplicate build source entries

Pull request author · upstream contribution

Opened 2025-07-18

Merged 2025-07-18

Merged upstream

The problem

The ELEMENTS_SOURCES list in lib/CMakeLists.txt repeated src/support/text_utils.cpp three times and receiver.cpp and theme.cpp twice each. The source manifest therefore contained redundant entries. [PR record](#)

Why it was needed

The build definition should identify each source once. Removing accidental duplication makes the list easier to inspect and maintain. Although the PR describes improved build efficiency and prevention of potential linking issues, it does not provide a failing build, measured speedup, or evidence that these duplicates actually caused repeated compilation. [PR record](#)

My contribution and the accepted change

Deleted four redundant lines from the existing source list: two additional text_utils.cpp entries and one each for receiver.cpp and theme.cpp. The original entries remain, so the cleanup preserves coverage of those translation units. No C++ implementation, source layout, library API, target relationship, or dependency was changed. [Changed files](#)

Why this approach

Documented rationale: remove duplicate source entries and keep the build functioning. Implementation-derived rationale: a direct list cleanup is sufficient because the issue is repeated declarations, not a missing component or platform-specific behavior. There is no reason to introduce new build logic to address this particular manifest inconsistency. [PR record](#) · [Changed files](#)

Validation and outcome

The author's PR body and commit message state that the build was verified after the change. No additional test, build log, environment matrix, or benchmark is part of the recorded patch. Build verification is author-reported; no performance measurements are supplied. Merged July 18, 2025. The concrete accepted outcome is a cleaner, deduplicated CMake source list. The review record contains no substantive design discussion beyond acceptance. [PR record](#) · [Changed files](#)

SOURCE RECORD

fix: Remove duplicate source file entries in CMakeLists.txt — pull request and author description — killerdevildog, 2025-07-18

Accepted implementation — changed files — cycfi/elements, 2025-07-18

Merge commit — djowel, 2025-07-18

STICKnoLOGIC/First-Accord · #55

Adding a community contributor profile

Community profile contributor

Opened **2025-07-12** Merged **2025-07-12****Contributor metadata accepted**

The problem

The contributor directory did not yet contain a profile record for killerdevildog. The PR body states simply that it adds the author to the contributor list; there is no reported application defect or engineering incident. [PR record](#)

Why it was needed

The contribution makes the author represented in this project's contributor collection. Its value is participation in the repository's contribution workflow and adding the data used for a profile. The change adds contributor-profile data. [PR record](#)

My contribution and the accepted change

Added contributors/killerdevildog.json, a 20-line structured record containing owner identity, GitHub and contact links, social entries, preferred learning resources, a description, GitHub-avatar preference, and display text. A maintainer commit updates and renames the initial this-is-killerdevildog.json to killerdevildog.json before merge. The final diff contains only the new JSON record. [Changed files](#)

Why this approach

Implementation-derived rationale: use the project's existing contributor-file shape so the profile can be consumed alongside other contributor records. The explicit PR description does not document deeper design alternatives or a technical motivation; the available record establishes the profile addition and review revisions. The rename demonstrates maintainer alignment with the accepted file naming. [PR record](#) · [Changed files](#) · [Discussion](#)

Validation and outcome

No executable tests, validation script, or application-code changes are recorded in the PR record. After merging, repository owner STICKnoLOGIC supplied a personalized First-Accord URL and congratulated the contributor. That is evidence of acceptance and a shared profile destination, not independent verification that the live page currently renders. Merged July 12, 2025. The accepted scope is community profile metadata. [PR record](#) · [Changed files](#) · [Discussion](#)

SOURCE RECORD

[Add killerDevildog to contriutor list](#) — pull request and author description — killerdevildog, 2025-07-12[Accepted implementation](#) — changed files — STICKnoLOGIC/First-Accord, 2025-07-12[Merge commit](#) — STICKnoLOGIC, 2025-07-12[Review discussion by STICKnoLOGIC](#) — STICKnoLOGIC, 2025-07-12

microsoft/vscode-copilot-release · #6998

Resolving a Copilot panel-placement request

Feature requester · workflow feedback

Opened **2025-03-27** Closed **2025-03-27**

Resolved through existing UI behavior

The problem

I wanted Copilot available on the left side of VS Code alongside Files, Search, Source Control, and Extensions. The report described the desired location and included VS Code 1.98.2, the installed extension version, and the Linux environment. [Issue report](#)

Why it was needed

The motivation recorded in the request was workspace consistency: access to Copilot in the same area as the other frequently used tools. This was a usability preference and discovery problem, not a demonstrated failure of the underlying code-completion system. [Issue report](#)

My contribution and the resolution

I filed a feature request explaining the desired interaction. Maintainer TylerLeonhardt responded that Copilot could already be moved there by dragging the Chat heading to the left. The issue was closed the same day. There is no associated merged implementation authored by me. [Issue report](#) · [Discussion](#)

Why this approach

The documented resolution uses existing panel-layout behavior, avoiding a new implementation when the requested placement is already supported. That rationale follows from the maintainer's instruction; it is not a design decision or code fix claimed as mine. The contribution is actionable user feedback and clarification of an available workflow. [Discussion](#)

Validation and outcome

The closing discussion gives a specific action to achieve the requested placement. GitHub marks the issue completed, but the evidence supports a support resolution rather than a shipped feature resulting from the request. Keeping that distinction makes the portfolio's account of contribution and ownership accurate. [Issue report](#) · [Discussion](#)

SOURCE RECORD

Original report · microsoft/vscode-copilot-release #6998 — Quaylyn Rimer / GitHub, 2025-03-27

Maintainer's drag-to-reposition guidance — TylerLeonhardt / Microsoft, 2025-03-27

`coder/code-server` · #5483

Diagnosing workspace paths containing a plus sign

Bug reporter · reproducible environment and steps

Opened **2022-08-20** Closed **2024-07-12**

Maintainer-confirmed resolution

The problem

Opening a folder whose name contained a plus sign, such as `dir_c++`, produced a workspace-does-not-exist message. My report described code-server 4.5.2, Firefox, an Ubuntu client, a Debian 11 i686 server, and an SSH-forwarded connection, with steps from folder creation to the failing dialog. [Issue report](#)

Why it was needed

A legal directory name should remain usable as a workspace path. The failure blocked opening affected folders rather than merely displaying their names incorrectly. The detailed environment and reproduction separated the visible symptom from an assumption about which layer was responsible. [Issue report](#)

My contribution and the resolution

I supplied the report, reproduction steps, expected behavior, actual dialog, and logs. Maintainer jsjoeio reproduced the problem on 4.6.0 but not in desktop VS Code, github.dev, or Codespaces, narrowing it to code-server. A later maintainer response said the latest version appeared to fix it. [Issue report](#) · [Discussion](#) · [Discussion](#)

Why this approach

The contribution was diagnostic: using a small filename change and comparing related products reduced the reproduction to an actionable compatibility case. The record does not identify the exact fixing commit or establish that I implemented a patch, so this page does not invent a URL-decoding algorithm or claim one as my design. [Issue report](#) · [Discussion](#) · [Discussion](#)

Validation and outcome

The issue was closed on July 12, 2024 after the maintainer's resolution statement. Independent reproduction is documented; the exact release and regression test that removed the failure are not linked in the issue timeline. The supported outcome is a reported defect later confirmed resolved by the project. [Discussion](#) · [Discussion](#)

SOURCE RECORD

Original report · `coder/code-server` #5483 — Quaylyn Rimer / GitHub, 2022-08-20

Independent reproduction and scope — jsjoeio / code-server, 2022-08-22

Maintainer resolution statement — code-asher / code-server, 2024-07-12